GPU Memory Optimization for DNN Training: A Comprehensive Survey of Past, Present, and Future

SHUIBING HE, Zhejiang University, China
NAN ZHANG, Zhejiang University, China
PING CHEN*, Zhejiang University, and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, China
WEIXU ZONG, Zhejiang Lab, China
YI ZHANG, Zhejiang University, China
SILING YANG, Zhejiang University, China

Deep neural networks (DNNs) have achieved significant success across various domains, but their rapid scaling in model size and training data volume has led to a severe GPU memory bottleneck. To address this issue, a variety of GPU memory optimization techniques have been proposed. However, these methods are often developed independently, making it difficult to systematically understand their trade-offs and applicability. Therefore, a systematic survey is needed to organize existing techniques and compare when they are effective, what overheads they introduce, and where they are applicable. In this survey, we present a comprehensive study of GPU memory optimization for DNN training. First, we analyze the main sources of GPU memory consumption and their scaling characteristics. Based on this, we introduce a unified taxonomy of existing approaches, categorizing them into five categories: data offloading, activation recomputation, data compression, distributed training, and other optimizations. Next, for each category, we describe the methods in detail, explain their underlying mechanisms, analyze the improvements in memory efficiency together with the computational and communication overheads they incur, and discuss their differences and potential combinations across categories. Finally, we discuss several emerging trends in GPU memory optimization for DNN training that are promising directions for future research.

1 INTRODUCTION

Deep neural networks (DNNs) have become a central driver of modern artificial intelligence, enabling significant advances in language understanding [158], computer vision [50], multimodal perception [125], and complex reasoning [170]. Over the past decade, DNN-based systems have been widely deployed in real-world applications, transforming industrial practices and accelerating the adoption of AI technologies.

Recent advances in DNN architectures, particularly Transformer architectures and their sparse variants such as mixture-of-experts (MoE) models [82, 158], have driven the development of models with dramatically larger parameter counts. Meanwhile, training workloads increasingly involve

*Corresponding author.

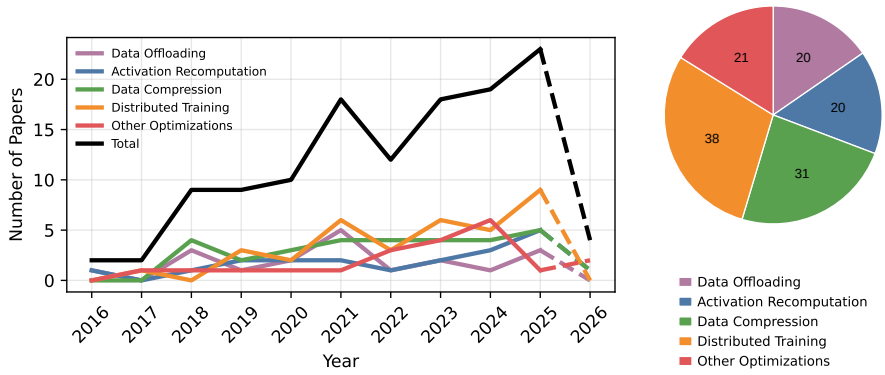
Authors' addresses: Shuibing He, heshuibing@zju.edu.cn, Zhejiang University, Hang Zhou, Zhejiang, China; Nan Zhang, Zhejiang University, Hang Zhou, Zhejiang, China; Ping Chen, zjuchenping@zju.edu.cn, Zhejiang University, and Hangzhou High-Tech Zone (Binjiang) Institute of Blockchain and Data Security, Hang Zhou, Zhejiang, China; Weixu Zong, Zhejiang Lab, Hang Zhou, Zhejiang, China; Yi Zhang, Zhejiang University, Hang Zhou, Zhejiang, China; Siling Yang, Zhejiang University, Hang Zhou, Zhejiang, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
0004-5411/2026/10-ART $15.00
https://doi.org/XXXXXXX.XXXXXXX

larger datasets, diverse data modalities, and longer input sequences, further increasing the GPU memory demand of DNN training. When GPU memory is insufficient, practitioners often have to reduce batch sizes or use smaller models, which can degrade training efficiency and limit model exploration. As a result, GPU memory capacity has become a key system bottleneck under practical hardware budgets.

To address this bottleneck, numerous GPU memory optimization techniques have been proposed in recent years. Many existing GPU memory optimization techniques are tailored to specific training scenarios or focus on certain optimization approaches. As the field continues to evolve, there is an increasing need for a comprehensive survey that systematically organizes existing GPU memory optimization techniques and examines them within a unified analytical framework.

Several existing surveys have made important efforts toward addressing this need, but they examine GPU memory optimization from different perspectives. Specifically, Tian et al. [154] investigate GPU memory optimization in the context of AI for Science models [69], with particular emphasis on scientific workloads. This application-oriented perspective offers valuable insights into scientific applications while leaving room for a more general survey of GPU memory optimization across a broader range of DNN training scenarios. Zhuang et al. [198], Wan et al. [161], Duan et al. [37], and Liang et al. [91] survey efficient DNN training from broader perspectives, covering multiple aspects such as computational efficiency, communication efficiency, memory utilization, and training scalability. Within these broader scopes, GPU memory optimization is discussed as one of several important aspects rather than being examined systematically as a dedicated topic. These existing surveys therefore leave room for a dedicated survey focused specifically on GPU memory optimization for DNN training.



(a) Temporal trends of research papers

(b) Category Distribution

Fig. 1. Overview of research in GPU memory optimization for DNN training. (a) Temporal trends of research papers from 2016 to 2026 (dashed lines indicate partial data up to June 2026). (b) Distribution of the five research categories.

Motivated by these needs, we present a systematic survey of GPU memory optimization that goes beyond a particular training scenario, analyzing a wide range of optimization techniques within a unified framework and examining their benefits, overheads, and applicability across diverse models, workloads, and hardware platforms. To establish the scope and analytical structure of the survey, we proceed in three stages. First, we collect representative GPU memory optimization papers published between 2016 and June 2026, mainly from leading systems and machine learning

conferences. We also include a small number of influential preprints from digital libraries such as arXiv, resulting in a final collection of 123 papers. The temporal distribution of these studies, as illustrated in Fig. 1(a), reveals steady growth in related publications, highlighting that GPU memory constraints have become an increasingly prominent bottleneck in DNN training.

Second, based on this collection, we analyze how existing methods manage GPU memory and develop a taxonomy according to their core optimization strategies. Specifically, we organize prior work into five distinct categories: *Data Offloading*, *Activation Recomputation*, *Data Compression*, *Distributed Training*, and *Other Optimizations,* the last of which encompasses techniques such as *GPU memory defragmentation* and *temporary buffer reduction.* As illustrated in Fig. 1, the temporal trends and statistical distribution of these studies demonstrate that substantial research efforts span all five categories, comprehensively covering the primary technical dimensions of GPU memory optimization.

Finally, beyond the high-level taxonomy, we provide a more detailed analysis of the trade-offs within each category. Specifically, we characterize the extra overheads introduced by different GPU memory optimization techniques, summarize the key challenges in reducing these overheads, and group existing studies according to their shared design principles. This analysis summarizes common design patterns and provides practical guidance for applying GPU memory optimization techniques to future DNN training workloads and emerging architectures.

The remainder of this survey is organized as follows. Section 2 introduces the fundamentals of DNN training and analyzes the GPU memory footprint to demonstrate the critical bottleneck of the GPU memory wall. Section 3 presents the taxonomy of memory optimization techniques for DNN training. Sections 4 – 8 systematically review Data Offloading, Activation Recomputation, Data Compression, Distributed Training, and Other Optimizations, respectively. Within each of these sections, we categorize existing approaches, explain their key techniques, analyze the overheads and design challenges they introduce, and further summarize common design principles and practical strategies for reducing them. Section 9 presents future research directions, and Section 10 summarizes the survey.

2 BACKGROUND

This section introduces deep learning models and standard training procedures (§2.1), analyzes the main sources of GPU memory consumption during training, including model parameters, optimizer states, intermediate activations, and other memory costs (§2.2), and discusses the challenges caused by the growing gap between GPU memory capacity and model scale (§2.3).

2.1 Fundamentals of Deep Neural Networks and Training Procedures

Deep Neural Networks (DNNs) are built from layers connected according to specific topological structures. At the architectural level, a DNN is mainly characterized by its layer types, network depth, and connectivity patterns. Different application domains have historically favored different layer designs. For example, early computer vision models [50] mainly use convolutional and pooling layers, while early natural language processing models [52] primarily use recurrent layers, such as long short-term memory (LSTM) layers. More recently, Transformer blocks [158], which mainly consist of self-attention and feed-forward layers, have become the dominant building blocks across both domains.

At the implementation level, these layers are further decomposed into fine-grained computational operators, such as matrix multiplication, softmax, and normalization. Therefore, a DNN can be represented as a directed computational graph [1, 117], where nodes denote operators and edges denote tensor dependencies and data flow. Among these operators, parameterized operators, such

as linear and convolution operations, contain trainable parameters that are iteratively updated during training.

Although DNNs have evolved into a diverse range of architectural forms, their training procedures still largely follow a common paradigm comprising three core stages: forward propagation, backward propagation, and parameter update, as shown in Fig. 2. During training, input samples are typically grouped into batches, and the number of samples in each batch is referred to as the batch size. In forward propagation, a batch of input samples is processed by the operators in the computational graph to generate model predictions. The generated intermediate activations are usually stored for gradient computation in backward propagation. In backward propagation, the loss is computed from the predictions and ground-truth labels, and gradients of activations and parameters are computed layer by layer using the chain rule. In parameter update, optimizers such as stochastic gradient descent (SGD) [135] and Adam [74] use these gradients to iteratively update model parameters.

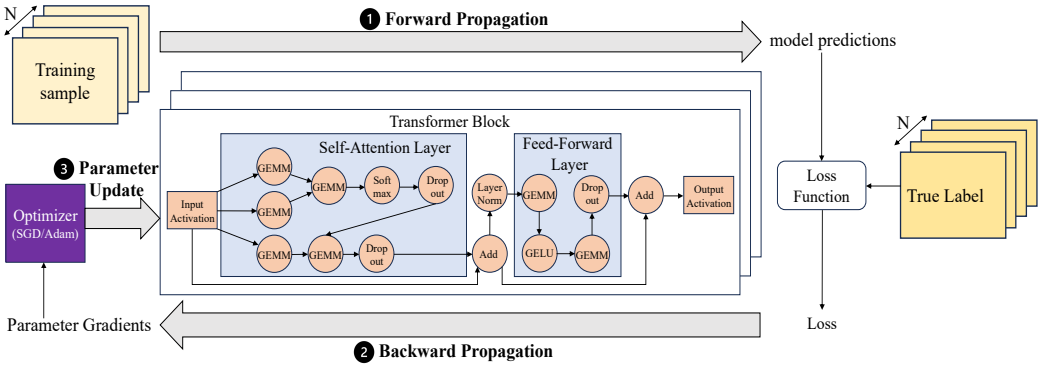


Fig. 2. Training workflow of DNNs.

2.2 Analyzing GPU Memory Consumption in DNN Training

During the training of DNNs, GPU memory consumption can be categorized into five primary sources:

Model Parameters. Model parameters refer to the learnable weights and biases associated with parameterized operators in DNNs, which are continuously updated through backpropagation and optimization algorithms. For a given network architecture, the number of model parameters remains fixed. Therefore, their memory footprint is independent of the batch size and depends primarily on the numerical precision format used for storage, such as FP16 or FP32. For a model containing M parameters, storing them in FP16 requires approximately $2M$ bytes of GPU memory.

Optimizer States. Optimizer states denote the auxiliary variables maintained by optimization algorithms to facilitate convergence and enhance training stability, often accounting for a substantial portion of the overall memory consumption. For instance, the Adam optimizer maintains first-order and second-order moments of the gradient for each parameter and, for a model containing M parameters, typically stores the corresponding momentum, variance, and a high-precision master copy in FP32 format, resulting in a total GPU memory footprint of approximately $12M$ bytes.

Gradients. Gradients generated during backpropagation include parameter gradients and activation gradients. Parameter gradients correspond one-to-one to model parameters and must be retained until the parameter update stage. For a model with M parameters, they require approximately $2M$ bytes in FP16 format. Activation gradients are temporary tensors used to propagate

gradients backward through the computational graph. Their memory footprint is determined by the size of intermediate activations, which depends on batch size, sequence length or input resolution, hidden dimension, and network depth.

Activations. Activations represent the intermediate results generated during forward propagation. During backpropagation, only the activations required for gradient computation need to be retained in GPU memory, while the remaining activations can be discarded. The GPU memory consumption of activations is determined by both the model architecture and the batch size. For a given model architecture, the activation memory footprint increases as the batch size grows.

Temporary Buffers and Memory Fragmentation. Temporary buffers refer to additional GPU memory allocated during training to support computation, communication, and other runtime operations, such as workspaces used by convolution [67] or matrix operations [42]. These buffers are typically short-lived and are released upon completion of the associated operations. Moreover, repeated GPU memory allocation and deallocation may give rise to GPU memory fragmentation [55, 143], which results in a substantial amount of scattered memory being occupied.

Table 1. Notation Definitions for Memory Analysis of the Transformer Architecture

M	Total number of trainable parameters in the model.
V	Vocabulary size.
L	Number of Transformer layers.
h	Hidden dimension size.
h_m	Feed-forward intermediate dimension (typically $h_m \approx 4h$).
n_h	Number of attention heads per layer.
b	Batch size.
s	Sequence length (number of tokens per input sample).
c_d	Memory cost per element of the dropout mask (in bytes).

To characterize GPU memory consumption, we consider FP16 mixed-precision training of Transformer models as a representative setting, which reflects the dominant architecture and training practice of modern large-scale models. In this setting, model parameters, gradients, and activations are stored in FP16 format, and Adam is used with FP32 optimizer states. The analysis assumes an idealized setting with a single GPU of effectively unlimited memory, ignoring distributed training and explicit GPU memory optimization techniques. Temporary buffers and memory fragmentation are not explicitly reflected in this example because they are stochastic and difficult to quantify, although they still incur additional memory overhead. The notations used in the analysis are summarized in Table 1.

Using these notations, we analyze the GPU memory footprint in detail and summarize the memory cost of each component in Table 2. First, the *model parameters* occupy $2M$ bytes and cover all trainable parameters in the Embedding, Attention, FFN, and language modeling (LM) head modules. Second, the *optimizer states* maintained by Adam require three FP32 tensors per parameter, accounting for $12M$ bytes. Third, the *gradient* memory includes only parameter gradients, which occupy $2M$ bytes. Activation gradients are temporary tensors generated during backpropagation and are released after use, so they are excluded from the GPU memory calculation. Finally, the *activations* consist of the intermediate results required for backpropagation in the Embedding, LayerNorm, Attention, FFN, and LM Head modules, and occupy $4L\,bs(4h+h_m+n_hs)+c_dL\,b(2sh+n_hs^2)+2bs(2V+h)$ bytes.

Building upon the preceding analysis, we present a concrete example to more clearly illustrate the composition of GPU memory consumption. Specifically, we estimate the memory composition of

Table 2. Memory components and their formulas (bytes).

Source	Components and Formula						
Model Parameters	Embedding					Vh	$2M = 4L\,h(2h + h_m) + 2Vh$ (FP16)
	Attention	Self-Attention	W_Q	$L\,h^2$			
			W_K	$L\,h^2$			
			W_V	$L\,h^2$			
		Linear Projection		$L\,h^2$			
	FFN	Up-Projection Matrix		$L\,hh_m$			
		Down-Projection Matrix		$L\,hh_m$			
	LM Head	None (Shared with Embedding)					
Optimizer States	Master Parameters	$4M = 8L\,h(2h + h_m) + 4Vh(FP32)$					
	First-Order Momentum	$4M = 8L\,h(2h + h_m) + 4Vh(FP32)$					
	Second-Order Variance	$4M = 8L\,h(2h + h_m) + 4Vh(FP32)$					
Gradients	$2M = 4L\,h(2h + h_m) + 2Vh(FP16)$						
Activations	Embedding	Input	$2bsV$				
	First LayerNorm	Input	$2L\,bsh$				
	Attention	Self-Attention	QKV Projection	Input	$2L\,bsh$		
			Dot-Product	Q	$2L\,bsh$		
				K	$2L\,bsh$		
			Softmax	Output	$2L\,bn_hs^2$		
			Dropout	Mask	$c_d\,L\,bn_hs^2$		
			Weighted Sum	Input	$2L\,bn_hs^2$		
				V	$2L\,bsh$		
		Linear Projection	Input	$2L\,bsh$			
		Dropout	Mask	$c_d\,L\,bsh$			
	First Residual	None					
	Second LayerNorm	Input	$2L\,bsh$				
	FFN	Up-Projection	Input	$2L\,bsh$			
		Activation Function (GELU)	Input	$2L\,bsh_m$			
		Down-Projection	Input	$2L\,bsh_m$			
		Dropout	Mask	$c_d\,L\,bsh$			
	Second Residual	None					
	LM Head	Output Projection	Input	$2bsh$			
		Softmax	Output	$2bsV$			
Total	$32L\,h(2h + h_m) + 16Vh + 4L\,bs(4h + h_m + n_hs) + c_dL\,b(2sh + n_hs^2) + 2bs(2V + h)$						

the Transformer model OPT-175B [188], which has approximately 1.75×10^{11} trainable parameters with $V = 50{,}272$, $L = 96$, $h = 12{,}288$, $h_m = 49{,}152$, and $n_h = 96$. Under the configuration of $b = 1$ and $s = 4096$, the memory required for *model parameters*, *optimizer states*, *gradients*, and *activations* is approximately **325 GiB**, **1950 GiB**, **325 GiB**, and **874 GiB**. Therefore, under ideal conditions and excluding temporary buffers and memory fragmentation, the total GPU memory required during training is estimated to be approximately **3.4 TiB**. This result highlights the substantial memory overhead of training large DNNs, with the total memory requirement exceeding that of model parameters alone by more than an order of magnitude (approximately **3.4 TiB** versus **325 GiB**).

2.3 GPU Memory Wall: A Key Bottleneck in DNN Training

GPUs constitute the primary hardware foundation for DNN training, and training performance depends on both computational capability and available on-device memory capacity. In recent years, GPU computing capability has advanced rapidly with the growth of parallel processing resources and specialized matrix-computation units, such as Tensor Cores [113]. By contrast, although GPU memory technologies have evolved from earlier GDDR-based designs to HBM [43], the growth of on-device memory capacity still lags behind the growth of computational throughput. At the same time, the GPU memory requirements of DNN training have increasingly outpaced the growth of physical GPU memory capacity. We refer to this widening supply and demand imbalance as the **GPU memory wall**, which increasingly constrains the scalability and efficiency of DNN training. Our analysis identifies three major factors that contribute to the continued widening of this gap:

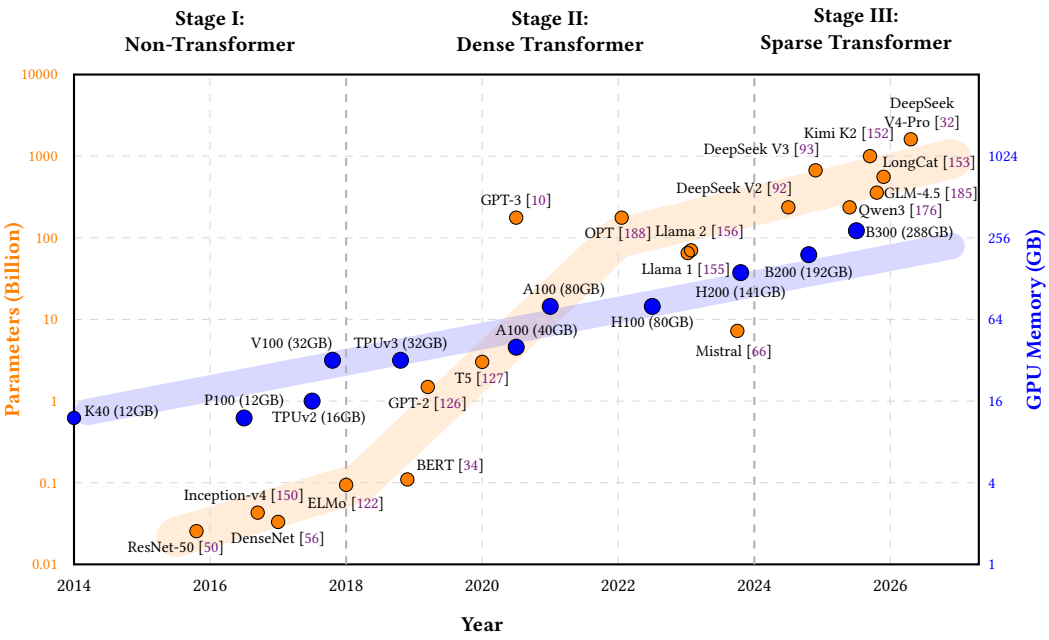


Fig. 3. Comparison of the growth trends in model parameter scales and GPU memory capacity.

- **Continuous Expansion of DNN Model Scale.** As scaling laws have gained prominence in the DNN field [71], model performance increasingly benefits from larger model scales, achieved by increasing network depth, width, and parameter count, as exemplified by Mixture of Experts (MoE) architectures. As shown in Fig. 3, model parameter counts have increased from 110M in BERT [34] to 1.6T in DeepSeek-V4 [32] between 2019 and 2026, representing an increase of approximately four orders of magnitude. This exponential growth in parameters demands vast GPU memory and has become a primary driver of the GPU memory wall.
- **Increasing Complexity of Training Data.** To enhance model capabilities, complex data forms such as long sequences [168] and multimodal data [57] are increasingly used in DNN training, which significantly intensifies GPU memory pressure. For long sequences, activations grow with sequence length and attention matrices grow quadratically. In multimodal models, multiple modality-specific encoders increase parameter counts, while modalities

like vision introduce large input tensors that further expand activations. As a result, these characteristics substantially increase the GPU memory footprint of DNN training.

- **Limited On-Device GPU Memory Capacity.** Despite continuous improvements in GPU memory technologies, the memory capacity available on a single GPU remains a fundamental constraint in DNN training, as shown in Fig. 3. As discussed in §2.2, under ideal conditions, training OPT-175B requires approximately 3.4 TiB of GPU memory, equivalent to the aggregate memory capacity of at least 20 NVIDIA B200 GPUs. This substantial mismatch between per device memory capacity and training memory demand is a key factor contributing to the GPU memory wall.

This bottleneck has severely constrained the development of DNNs. First, to avoid out-of-memory (OOM) failures, practitioners are often compelled to adopt more conservative training strategies, such as reducing the batch size [20], which may lead to underutilization of GPU computational resources and thereby prolong the training process. Second, the GPU memory wall directly constrains the sequence length and data complexity required for training. However, training data is a fundamental determinant of model quality, and limitations on sequence length and data complexity may therefore adversely affect the performance of the resulting model [9]. Third, the high memory demand of DNN training substantially increases hardware costs, making large-scale training accessible only to organizations with sufficient computing resources. This raises the entry barrier for model development, limits experimental exploration, and slows down the iteration of new model architectures and training strategies.

3 TAXONOMY OF GPU MEMORY OPTIMIZATION TECHNIQUES

To overcome the limitations imposed by the GPU memory wall, a wide range of optimization techniques has emerged over the past decade. In this survey, we systematically organize existing methods by their underlying optimization mechanisms and classify them into five coherent categories, including four mainstream memory optimization techniques and one category of other complementary optimizations, as follows:

(1) **Data Offloading Techniques.** When GPU memory is insufficient, part of the data is offloaded to external devices, such as CPU main memory and SSDs, as discussed in §4.
(2) **Activation Recomputation Optimization.** Activations are not saved during the forward pass but are recomputed on demand during the backward pass to reduce memory usage, as discussed in §5.
(3) **Data Compression Techniques.** Data are compressed to reduce the required storage space and decompressed before use, as discussed in §6.
(4) **Distributed GPU Memory Optimization.** The model and training data are partitioned across multiple GPUs to distribute memory pressure, as discussed in §7.
(5) **Other Optimizations.** This category covers innovative techniques for reducing memory overhead, such as mitigating GPU memory fragmentation and reducing temporary buffer usage, as discussed in §8.

Beyond a high-level classification of techniques, we further identify the specific challenges faced by each category and provide a fine-grained taxonomy of existing solutions to these challenges. The overall taxonomy is illustrated in Fig. 4.

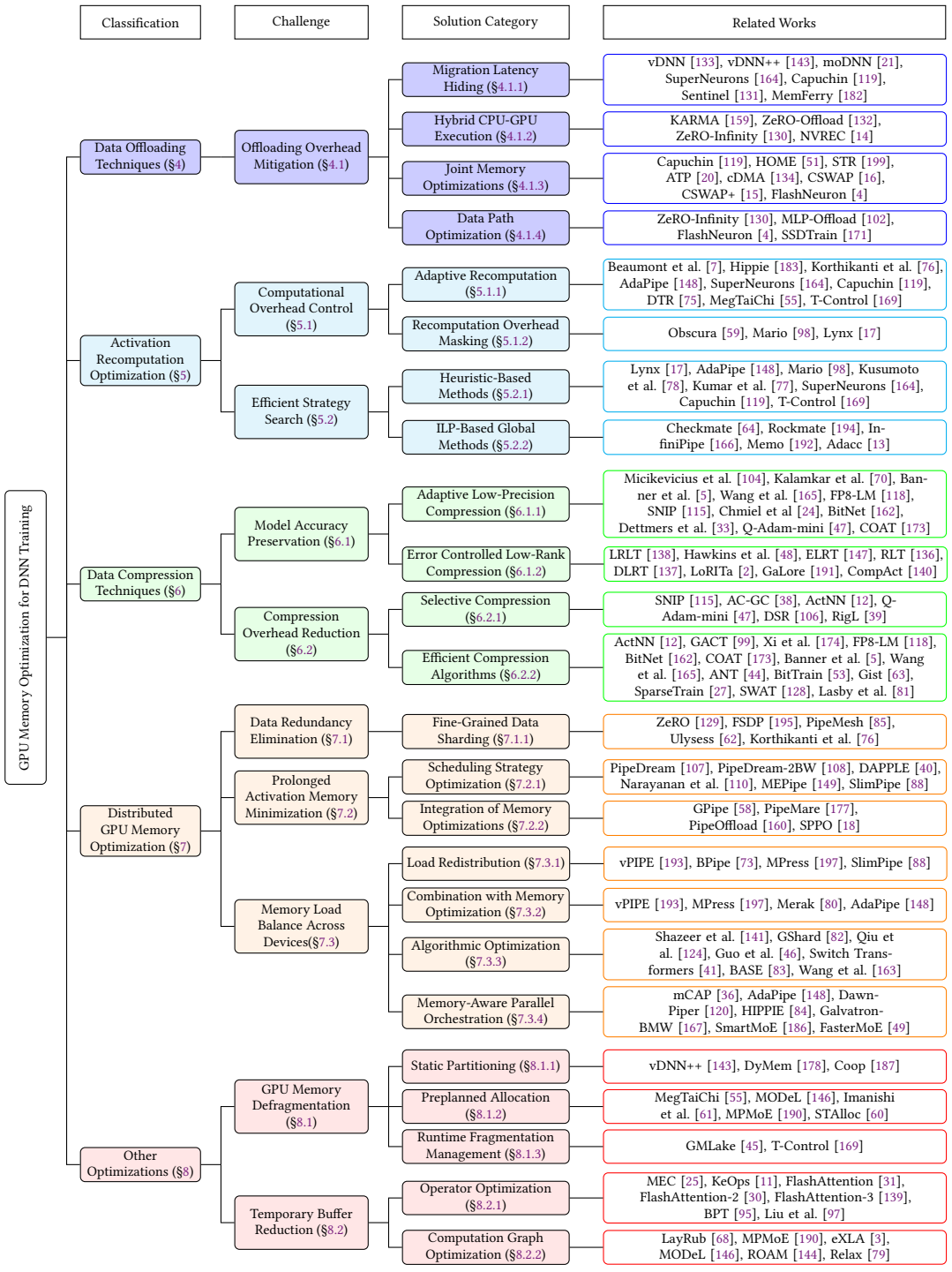


Fig. 4. Overview of memory optimization techniques

4 DATA OFFLOADING TECHNIQUES

Offloading alleviates the GPU memory bottleneck during DNN training by migrating various types of data from GPU memory to external memory tiers such as CPU DRAM or SSDs, as illustrated in Fig. 5. Specifically, this paradigm strategically offloads inactive data to external storage and prefetches it back on demand for subsequent computations. Nevertheless, transferring data across memory tiers inherently incurs substantial I/O overhead, making the reduction of data migration cost a critical challenge.

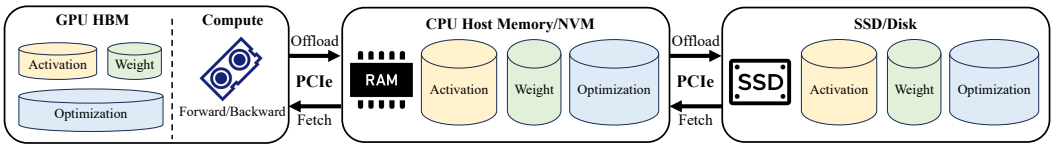
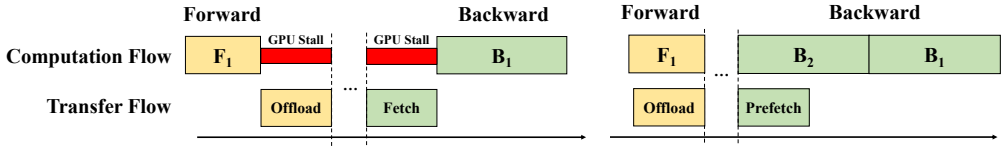


Fig. 5. Offloading system for DNN training.

4.1 Offloading Overhead Mitigation

Fundamentally, data offloading relies on system interconnects such as PCIe to transfer tensors across memory tiers. However, these interconnects offer bandwidths that are orders of magnitude lower than those of on-device memory and exhibit significantly higher latencies, resulting in substantial data migration overhead. This I/O bottleneck frequently leaves the GPU waiting for required data, resulting in frequent GPU stalls and a substantial reduction in end-to-end training efficiency, as illustrated in Fig. 6(a).

To mitigate this overhead, extensive research has proposed various strategies, which can be broadly classified into four categories: **(1) migration latency hiding**, **(2) hybrid CPU-GPU execution**, **(3) joint memory optimizations**, and **(4) data path optimization**.



(a) Default Offloading (b) Migration Latency Hiding

Fig. 6. Execution timeline comparison for offloading scheduling. (a) Default Offloading incurs GPU stalls that interrupt model computation; (b) Migration Latency Hiding mitigates stalls by parallelizing migration and computation.

4.1.1 ***Migration Latency Hiding.*** The core principle of this approach is to overlap data migration with forward and backward computations, thereby hiding substantial I/O latency, as illustrated in Fig. 6(b). Achieving this requires effective scheduling strategies, which determine when to offload data to reclaim GPU memory while ensuring its availability before consumption, thus preventing migration latency from stalling the critical path.

vDNN [133] pioneered dual CUDA streams to overlap data migration with computation. Specifically, it offloads the current layer's activations to CPU DRAM during forward propagation and prefetches the preceding layer's activations during backward propagation, using synchronization

points to ensure the completion of respective memory migrations. However, vDNN's strict synchronization points restrict the overlap of data migration and computation to within a single layer's execution window. vDNN++ [143] mitigates this bottleneck via a conditional stalling mechanism that utilizes CUDA events to monitor data dependencies. It enables immediate computation of subsequent layers as long as GPU memory and required data are ready, thereby maximizing the overlap between data migration and computation without unnecessary synchronization. On the other hand, MemFerry [182] leverages GPU Direct Host Access (DHA) to compute directly on CPU DRAM, avoiding HBM consumption from frequent parameter migrations. It strategically exploits the extended computation window of DHA to proactively overlap with the migration of other layers' data through PCIe.

Despite these advancements, the migration strategies in these early works remain relatively rigid. They operate strictly at a layer-level granularity, which prevents the fine-grained selection of specific tensors for offloading and the dynamic optimization of migration timing. Recognizing this bottleneck, subsequent works have introduced more flexible mechanisms to overcome these limitations. moDNN [21] profiles computational and migration latencies prior to training and leverages its scheduler to perform an offline static simulation of the entire training process, utilizing a heuristic algorithm to generate a data migration schedule that minimizes total iteration time, precisely specifying the tensors and timing for offloading and prefetching. SuperNeurons [164] exploits the temporal locality of tensor accesses during backpropagation by introducing an LRU cache at the tensor level on the GPU. This mechanism retains hot tensors locally while offloading only cold tensors to the CPU, minimizing the overhead of migrating data over PCIe.

Capuchin [119] makes tensor swapping decisions based on runtime tensor access patterns rather than static layer types. It first profiles tensor accesses in an initial iteration and identifies candidate tensors that are accessed multiple times and whose lifetimes overlap the peak-memory period. Among these candidates, it prioritizes tensors with long reuse intervals, using Free Time (FT) to estimate whether swap-out and prefetching can be hidden by normal computation. Tensors with larger FT are preferred for swapping, while prefetch triggers are adjusted dynamically using runtime feedback. When swapping cannot sufficiently reduce memory pressure or introduces excessive overhead, Capuchin falls back to alternative memory-saving strategies. Sentinel [131] reveals that prior offloading schemes ignore OS memory management at the page level for CPU DRAM, causing tensors with divergent access frequencies to share pages and trigger redundant migrations. To address this, it introduces data reorganization at the tensor level to group tensors with similar lifetimes into unified pages, alongside an analytical model for dynamic migration scheduling.

4.1.2 ***Hybrid CPU-GPU Execution.*** This method breaks the conventional assumption that training computations must be strictly confined to the GPU. Since offloaded tensors already reside in host memory, it schedules specific computations directly to the CPU, eliminating redundant data migrations across memory tiers to mitigate the performance degradation caused by offloading, while additionally leveraging idle CPU resources, as shown in Fig. 7.

KARMA [159] maintains model parameters and optimizer states resident in CPU DRAM, dynamically swapping parameters and activations between the CPU and GPU at block granularity. Upon completing a block's backward pass, its gradients are immediately offloaded to CPU DRAM, enabling the CPU to execute phased AllReduce communication and parameter updates. In contrast, ZeRO-Offload [132] extends ZeRO-2 (detailed in §7.1.1) by partitioning parameters across GPUs. Each GPU stores only its assigned parameter shard, while the corresponding optimizer states are kept in CPU DRAM. During backpropagation, GPUs compute gradients and perform ReduceScatter to obtain their gradient shards. The gradient shards are then moved to CPU memory, where the

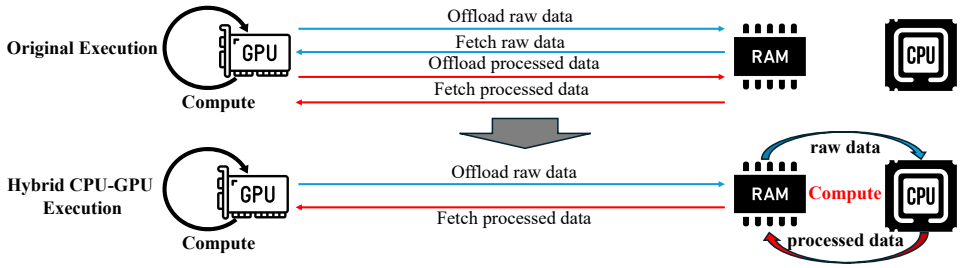


Fig. 7. Hybrid CPU-GPU Execution reduces the number of data migrations by scheduling computations directly on the CPU.

CPU updates the corresponding parameter shards using the resident optimizer states. By keeping only parameter shards on GPUs and offloading optimizer states to the CPU, ZeRO-Offload reduces GPU memory consumption and avoids frequent migrations of full optimizer states. Inheriting the idea of utilizing CPU for optimizer updates from ZeRO-Offload, ZeRO-Infinity [130] leverages the data partitioning technique of ZeRO-3 (detailed in §7.1.1) and further introduces the NVMe SSD storage tier to dynamically offload parameters, gradients, and optimizer states to CPU memory or NVMe SSD.

Similarly, NVREC [14] exploits CPU-side non-volatile memory (NVM) to offload part of the model parameters in embedding-intensive recommendation models. It places cold embedding vectors in CPU-side NVM while keeping hot vectors in DRAM to reduce access latency. During training, the CPU handles embedding lookup/update and feature interaction, while the GPU executes MLP computation, enabling efficient CPU–GPU collaborative training.

4.1.3 ***Joint Memory Optimizations.*** To mitigate the communication bottlenecks caused by frequent data migrations in offloading, recent approaches often combine offloading with other memory optimization techniques to reduce the volume of transferred data. According to the complementary technique used, these joint strategies can be broadly categorized into two classes:

(1) Offloading with Recomputation. This approach replaces offloading with recomputation for selected tensors when recomputing them is cheaper than transferring them. In this way, it reduces unnecessary data migrations and communication overhead, while still relieving memory pressure with only limited extra computation.

Capuchin [119] prioritizes offloading, but evaluates recomputation when a tensor's free time window is insufficient to fully hide the swapping overhead. If recomputation incurs lower overhead, Capuchin dynamically switches to it, thereby minimizing the global performance penalty. Additionally, several GPU memory management frameworks, such as HOME [51], STR [199], and ATP [20], formulate joint tensor placement as a combinatorial optimization problem under strict memory constraints to optimize training efficiency. Specifically, these systems determine whether to retain, offload, or recompute each tensor within the computational graph to derive an optimal execution plan. By leveraging efficient solvers, they systematically identify policies that minimize iteration latency and maximize training throughput under a fixed GPU memory budget.

(2) Offloading with Compression. This strategy reduces offloading overhead by compressing tensors before they are transferred and decompressing them when they are fetched back. Since less data needs to move across the memory hierarchy, the communication cost of offloading is reduced.

Exploiting the inherent high sparsity of activation maps in DNN training, cDMA [134] introduces a lightweight Zero-Value Compression hardware logic. This unit is directly embedded alongside the GPU's memory controller, thereby significantly reducing the PCIe communication overhead

associated with data offloading and prefetching processes. Unlike cDMA that relies on customized hardware, CSWAP [16] and CSWAP+ [15] propose a purely software-based tensor compression offloading framework. This framework directly leverages GPU compute units to perform compression and decompression. Driven by dynamic cost-effectiveness evaluation, the system uses lightweight models to predict time overhead, triggering compression offloading only when the combined compression and migration latency is strictly lower than the original migration time. This prevents the performance degradation of indiscriminate static compression, thereby improving overall training efficiency. Targeting SSD offloading, FlashNeuron [4] analyzes the computational graph to selectively identify highly compressible tensors whose migration latency is strictly bounded by computation latency, subsequently compressing them via a customized sparse matrix format to ensure I/O overhead is fully overlapped with GPU computation.

4.1.4 ***Data Path Optimization.*** Standard offloading suffers from slow data migrations due to PCIe bandwidth limits and software I/O overhead. To address this problem, existing work improves data migration from two complementary perspectives: (i) aggregating multiple data migration paths across memory tiers, and (ii) optimizing individual data migration path. These approaches either increase aggregate bandwidth or improve single-path efficiency.

(1) Multi-Path Data Migration Optimization. This method maximizes interconnect bandwidth utilization and mitigates I/O contention between the GPU and external memory tiers through advanced concurrency control and scheduling strategies, thereby enhancing data migration efficiency and reducing the performance overhead associated with data offloading.

Existing offloading systems often fail to fully utilize the available bandwidth across GPUs, PCIe interconnects, CPU memory, and storage devices, resulting in substantial resource underutilization. To address this limitation of underutilized system bandwidth and resources, ZeRO-Infinity [130] introduces a bandwidth-centric partitioning mechanism that hierarchically distributes model states across GPU memory, CPU DRAM, and NVMe SSDs. It partitions parameters across data-parallel devices, allowing each GPU to directly fetch its assigned parameter shard from CPU DRAM or NVMe SSDs, replacing centralized data migration with parallel I/O and improving aggregate bandwidth utilization.

To further address contention in shared storage and interconnects, ZeRO-Infinity observes that concurrent access from multiple GPUs can overload PCIe links and NVMe controllers, leading to performance instability [101]. It mitigates this issue using the DeepNVMe engine, which improves SSD request handling via batched asynchronous I/O and optimized scheduling. In contrast, MLP-Offload [102] tackles the same memory hierarchy bottleneck through system-level orchestration. It introduces lightweight node-level locking and a virtual multi-path tiering mechanism to coordinate access and dynamically balance data placement between local NVMe SSDs and remote parallel file systems, reducing contention and improving overall throughput.

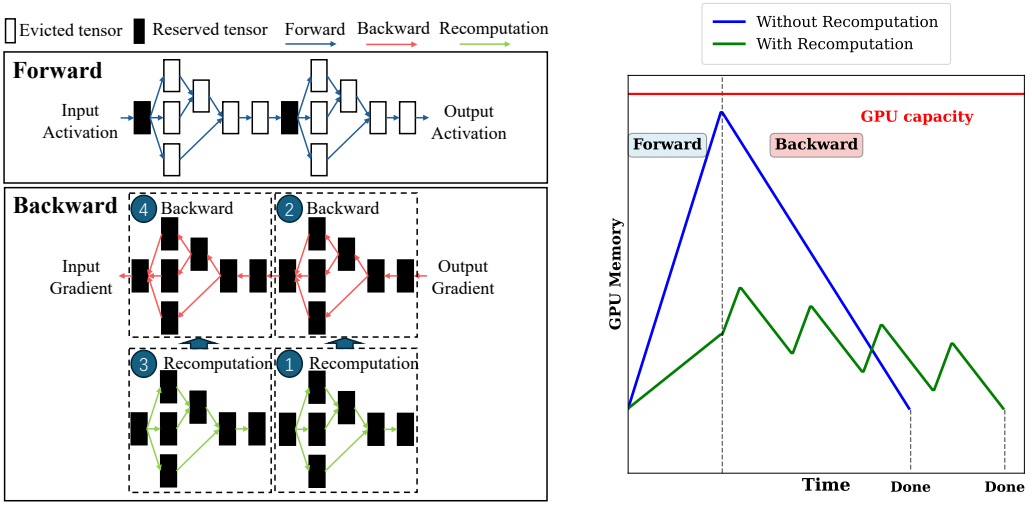
(2) Novel Migration Pathway Optimization. This approach leverages the direct access capabilities of the underlying hardware to reconstruct the system I/O stack, establishing a direct peer-to-peer (P2P) data pathway between the GPU and external storage. This effectively eliminates redundant intermediate memory copies in conventional data paths, thereby significantly reducing cross-tier migration latency.

FlashNeuron [4] leverages NVIDIA GPUDirect-based GDRCopy along with Intel SPDK to enable direct P2P data migration between the GPU and NVMe SSDs via DMA. This customized user-space I/O software stack completely bypasses intermediate memory copies in the CPU DRAM. With the rapid evolution of the system software ecosystem, SSDTrain [171] directly adopts the standardized NVIDIA GPUDirect Storage (GDS) [112] framework. By establishing a standardized direct GPU-SSD

data channel via the GDS driver, this work further eliminates the reliance on CPU buffers, thereby significantly reducing cross-tier I/O migration latency and improving overall training efficiency.

5 ACTIVATION RECOMPUTATION OPTIMIZATION

Recomputation (a.k.a. gradient checkpointing or re-materialization) reduces memory footprint during DNN training by selectively discarding intermediate tensors in the forward pass and regenerating them on demand in the backward pass, as illustrated in Fig. 8(a). Instead of materializing and retaining all intermediate results for gradient computation, the training procedure stores only a subset of checkpoint tensors at carefully chosen locations. During backpropagation, missing tensors between checkpoints are recomputed by partially re-executing the corresponding segments of the forward graph.



(a) Forward-Backward recomputation workflow

(b) Impact of recomputation on training

Fig. 8. Schematic illustration of activation recomputation and its impact on training. (a) Forward-Backward recomputation workflow; (b) Impact of recomputation on training. Recomputation reduces peak GPU memory consumption at the cost of introducing additional computational overhead.

Early recomputation works focus on small-scale DNNs, like CNNs for image classification and RNNs for language modeling. Chen et al. [19] first proposed applying recomputation to DNN training. It partitions the model into a small number of equal-length segments and performs recomputation within each segment. Recent research focuses on large-scale Transformer-based LLMs. Recomputation optimizes GPU memory usage, enabling larger batch sizes, longer context lengths, or deeper models under fixed memory budgets. Nevertheless, this approach brings two critical issues that impede its practical deployment and overall training efficiency:

- **Increase in Training Iteration Time.** This challenge stems from the extra computation introduced by recomputing activations during the backward pass.
- **Inefficient Search for Optimal Policies.** Recomputation must decide which tensors to discard and recompute to balance memory savings and computation overhead. This challenge arises because the search space is large, with different operators offering different recomputation costs and memory-saving benefits.

5.1 Computational Overhead Control

Recomputation necessitates re-executing forward computations during backpropagation. The full recomputation strategy discards nearly all intermediate activations, which minimizes peak memory consumption to the greatest extent but incurs an additional complete forward pass per training iteration, as illustrated in Fig. 8(b). However, overly conservative recomputation policies fail to adequately alleviate memory pressure. Therefore, it is crucial to control the associated computational overhead when leveraging recomputation for memory optimization. To achieve this objective, existing works have proposed two categories of approaches: **(1) adaptive recomputation** and **(2) recomputation overhead masking**.

5.1.1 ***Adaptive Recomputation.*** Adaptive recomputation selects which intermediate tensors to discard and when to recompute based on their memory-compute trade-offs. This paradigm encompasses two primary methodologies: static and dynamic adaptive recomputation.

(1) Static adaptive recomputation. These methods determine the recomputation strategy before training begins and keep the strategy fixed throughout training. Operationally, a common pattern across these methods involves first profiling the memory footprint and computational cost of individual layers or operators, followed by constructing a cost model to guide recomputation decisions. Specifically, Beaumont et al. [7] and Hippie [183] formulate recomputation strategies at the layer granularity. For each layer, they adopt a binary policy: either all activations are preserved throughout training, or the entire layer is recomputed on demand, without decomposing operations within the layer. Korthikanti et al. [76] conduct selective recomputation at the operator granularity by manually identifying memory-dominant yet compute-light operators, such as attention mechanisms. Alternatively, AdaPipe [148] constructs a cost model based on the memory pressure of each stage within pipeline parallelism, automatically searching for operator-level recomputation strategies that align with the memory characteristics of respective stages.

(2) Dynamic adaptive recomputation. These methods employ a runtime-aware mechanism to determine recomputation policies online during training. Operationally, such approaches continuously monitor real-time memory pressure alongside the actual memory footprint and recomputation overhead of individual tensors, dynamically adjusting the retention and eviction strategies for intermediate activations. Theoretically, compared with static recomputation schemes adopting fixed schedules, this dynamic paradigm exhibits better adaptability in processing variable-length inputs and dynamic DNN architectures such as Mixture-of-Experts models. Specifically, SuperNeurons [164] and Capuchin [119] respectively implement dynamic recomputation at the layer and operator granularities. DTR [75] and MegTaiChi [55] trigger adaptive recomputation when memory pressure nears the OOM threshold, selecting eviction candidates based on a set of heuristic metrics including tensor staleness, tensor size, and recomputation cost. Furthermore, T-Control [169] highlights the critical impact of recursive rematerialization depth in the tensor dependency graph on the efficiency of the training process. It accounts for this depth when selecting checkpoint tensors, thereby further reducing recomputation overhead.

5.1.2 ***Recomputation Overhead Masking.*** Distributed training often contains GPU stalls from two main sources, as illustrated in Fig. 9. The first source is pipeline-parallel stalls, including warm-up stalls before the pipeline reaches the steady state and P2P communication stalls between adjacent pipeline stages. The second source is tensor-parallel communication stalls, where GPUs wait for collective communication results before continuing computation. Recomputation Overhead Masking aims to schedule recomputation within these stall intervals, thereby hiding part of the recomputation cost and reducing its impact on training efficiency.

Pipeline-parallel stalls. Obscura [59] and Mario [98] hide recomputation overhead within pipeline-parallel stalls, as shown in Fig. 9(a). These stalls come from both the warm-up phase and P2P communication between adjacent pipeline stages. Obscura advances the original forward computations of certain pipeline stages to enlarge the idle intervals that can later be used for recomputation forward passes. Mario further optimizes the pipeline schedule by overlapping recomputation forward passes with pipeline stalls, removing unnecessary recomputation.

Tensor-parallel communication stalls. Lynx [17] targets communication stalls in tensor parallelism, as depicted in Fig. 9(b). It overlaps recomputation with AllReduce communication, allowing recomputation to proceed while GPUs wait for tensor-parallel communication results.

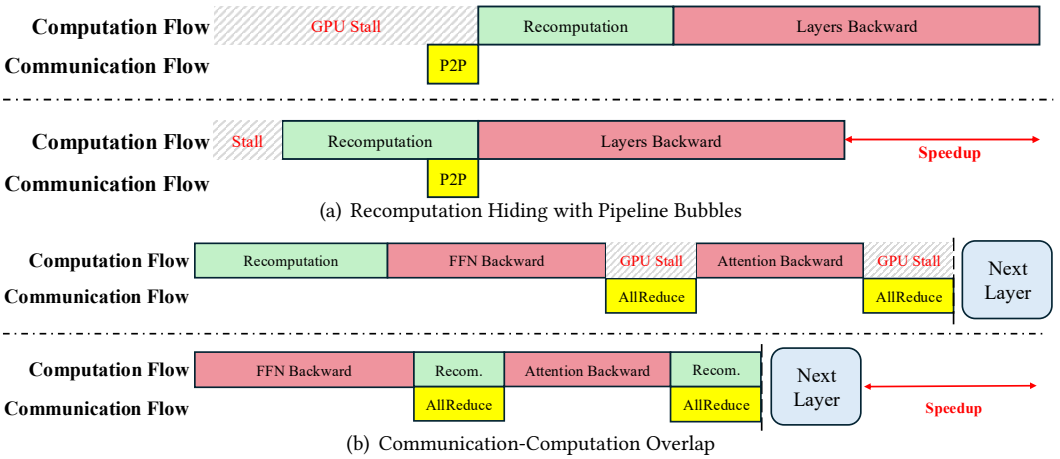


Fig. 9. Recomputation Overhead Masking. (a) Hiding the overhead of recomputation within the idle bubbles of pipeline scheduling; (b) Overlapping recomputation with communication operations. For each subfigure, the upper part presents the naive scheme, while the lower part shows the optimized scheme.

5.2 Efficient Strategy Search

Early works [8, 19, 76] derive recomputation strategies from hand-crafted rules that preselect activations without explicit search, reducing policy-generation overhead but often yielding sub-optimal training performance across different models and hardware. As recomputation becomes configuration-dependent, search emerges as a central challenge. For offline schemes, long search time delays training and results are not portable across configurations. For online schemes, decisions must be updated during training, imposing strict latency constraints. Moreover, the search space is combinatorial, spanning tensor types, dependencies, and distributed settings, making exhaustive search infeasible. Therefore, effective methods must balance search cost and solution quality. Existing approaches can be broadly categorized into **Heuristic-Based Methods** and **ILP-Based Global Methods**.

5.2.1 ***Heuristic-Based Methods.*** These methods approximate the ideal objective of minimizing total recomputation cost under a strict memory budget. We consider a set of intermediate tensors $\mathcal{T}$ produced during training. For each tensor $t \in \mathcal{T}$, we denote $m_\tau(t)$ as the memory usage contributed by t at time τ if t is kept materialized. Let $\mathcal{R} \subseteq \mathcal{T}$ denote the set of tensors that are chosen for

recomputation. The peak memory usage at time τ under a given recomputation set $\mathcal{R}$ is then

$$M_\tau(\mathcal{R}) \;=\; \sum_{t\in\mathcal{T}\setminus\mathcal{R}} m_\tau(t) \tag{1}$$

Given memory budget M_{budget}, an idealized formulation of the recomputation planning problem is

$$\min_{\mathcal{R}\subseteq\mathcal{T}} \quad \sum_{t\in\mathcal{R}} c(t) \tag{2}$$

$$\text{s.t.} \quad \max_{\tau}\; M_\tau(\mathcal{R}) \;\le\; M_{\text{budget}} \tag{3}$$

where $c(t)$ denotes the additional compute cost of recomputing t. In practice, heuristic-based methods do not solve (2)–(3) exactly. Instead, they approximate this objective by assigning to each tensor t a heuristic score $h(t)$, and then prefer to discard tensors with lower $h(t)$ when the memory budget would otherwise be exceeded. Each tensor's heuristic score $h(t)$ depends on its memory-compute characteristics. A general form is

$$h(t) \;=\; f\big(m(t),\, c(t),\, u(t),\, s(t)\big), \tag{4}$$

where $f(\cdot)$ is a lightweight scoring function, $m(t)$ denotes the memory usage of tensor t, $c(t)$ denotes the recomputation cost of t, $u(t)$ denotes an optional measure of importance (e.g., sensitivity or reuse frequency), and $s(t)$ denotes schedule-related features (e.g., stage index, bubble or communication regions). A common instantiation is to demonstrate $h(t)$ is

$$h(t) \;=\; \frac{c(t)}{m(t)} \cdot \phi\big(u(t)\big) \cdot \psi\big(s(t)\big), \tag{5}$$

where $\phi(\cdot)$ and $\psi(\cdot)$ reweight tensors according to expected future reuse or their position in the pipeline schedule. Heuristic policies then use $h(t)$ to approximate the optimal recomputation set. For example, one can define

$$\mathcal{R} \;=\; \{\, t \in \mathcal{T} \mid h(t) \le \tau_h \,\}, \tag{6}$$

for some threshold τ_h, or, in an online setting, when the memory usage would exceed M_{budget}, iteratively evict tensors with the smallest scores:

$$t^\star \;=\; \arg\min_{t\in\mathcal{T}_{\text{resident}}} h(t), \tag{7}$$

and mark $t^\star$ for recomputation. In this way, heuristic methods approximate the ideal objective using cheap, local decisions while respecting the global memory budget.

Recent works adopt heuristic-based methods following this pattern, and tailor the heuristic algorithms to their specific training contexts. Lynx [17] combines an optimization model with a scalable heuristic to overlap recomputation with communication and idle intervals. Both AdaPipe [148] and Mario [98] utilizes lightweight dynamic programming to minimize end-to-end time with given memory budgets. Kusumoto et al. [78], Kumar et al. [77] build arbitrary computation graphs where vertices are tensors and edges are ops, and use dynamic programming over a topological order plus clever pruning to find the globally optimal checkpoint subset under a peak-memory objective. SuperNeurons [164] and Capuchin [119] profile tensor access patterns dynamically and use greedy heuristics to trigger recomputation. T-Control [169] proposes an incremental heuristic value update algorithm to efficiently construct dynamic adaptive recomputation policies.

5.2.2 ***ILP-Based Global Methods.*** These methods treat recomputation planning as a global optimization problem, formally casting it as an Integer Linear Program (ILP). Specifically, they encode the forward-backward execution graph into binary decision variables and linear constraints, enabling solvers to systematically explore the combinatorial solution space and guarantee globally optimal schedules.

A unified ILP template can be expressed as follows. We consider a set of tensors $\mathcal{T}$ and a discrete set of execution steps $\mathcal{S}$. For each tensor $t \in \mathcal{T}$, we denote its computational cost by c_t and its memory footprint by m_t. ILP-based methods typically introduce the following binary decision variables:

- $x_{t,s} \in \{0, 1\}$: indicates whether tensor t is computed at step s;
- $k_{t,s} \in \{0, 1\}$: indicates whether tensor t is retained in memory immediately after step s.

The objective is to minimize the total computational cost including recomputation, formulated as:

$$\min_{x,k} \quad \sum_{s \in \mathcal{S}} \sum_{t \in \mathcal{T}} c_t \, x_{t,s}. \tag{8}$$

A peak-memory constraint is imposed at every execution step:

$$\sum_{t \in \mathcal{T}} m_t \, k_{t,s} \leq M_{\text{budget}}, \qquad \forall s \in \mathcal{S}. \tag{9}$$

To preserve computation graph correctness, dependency constraints ensure that all predecessors are available before a tensor is computed:

$$x_{t,s} \leq k_{u,s-1}, \quad \forall u \in \text{pred}(t), \, \forall s \in \mathcal{S}, \, s > 1, \tag{10}$$

where $\text{pred}(t)$ denotes the set of tensors required to compute t. Finally, each tensor must be produced at least once, either during the initial forward pass or via recomputation:

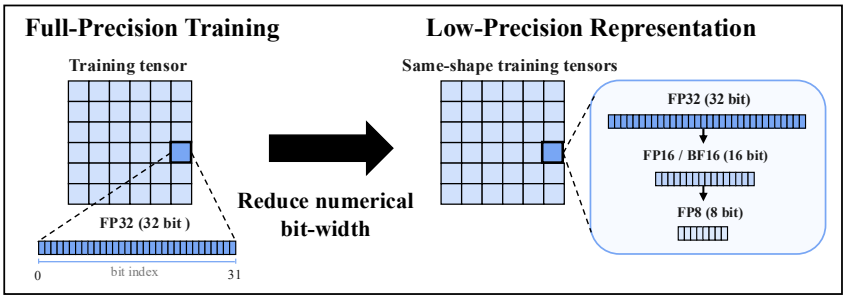
$$\sum_{s \in \mathcal{S}} x_{t,s} \geq 1, \qquad \forall t \in \mathcal{T}. \tag{11}$$

Together, (8)–(11) form a generic ILP formulation for globally optimal recomputation scheduling. Concrete systems extend this template in different ways, such as introducing tensor-lifetime variables or incorporating pipeline parallelism.

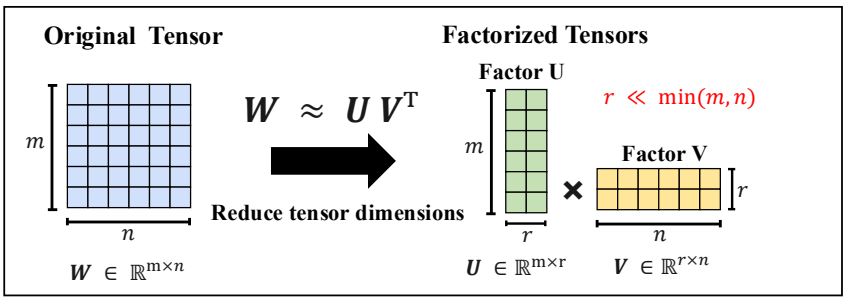
Accordingly, existing works primarily focus on how to efficiently solve the resulting ILP formulations. They differ mainly in how they relax or decompose the model to keep the solving overhead acceptable. Checkmate [64] accelerates ILP solving by exploiting the structure of neural network DAGs. It reduces variable count through tensor-lifetime compression and pruning infeasible execution orders before constructing the ILP. Rockmate [194] decomposes the global ILP into small block-level subproblems and then stitches the solutions using dynamic programming, dramatically reducing solving complexity. InfiniPipe [166] improves ILP solving efficiency by assigning ILP only to pipeline-stage and microbatch scheduling decisions while leaving intra-stage details to analytical models, resulting in a hybrid planner that dramatically reduces ILP size. There are also several recent works [13, 192] that apply standard ILP-based methods for their recomputation search. Though not as efficient as heuristic methods at runtime, ILP provides globally optimized and highly reliable recomputation schedules that capture long-range dependencies and complex resource interactions.

6 DATA COMPRESSION TECHNIQUES

Many conventional compression techniques fall outside this scope. For example, pruning [65, 179, 180] and post-training quantization [189] mainly reduce model size; knowledge distillation [100] trains smaller student models; gradient compression [105, 123] primarily reduces communication traffic. Since these techniques mainly target model compactness or communication efficiency rather than treating peak training memory as the primary optimization objective, they are not the main focus of this paper. Instead, we focus on training-time compression methods that store key training tensors in more compact forms. Existing studies adopt several representative approaches, as outlined below.



(a) Tensor Bit-Width Reduction



(b) Tensor Dimensionality Reduction

Fig. 10. Overview of data compression techniques for memory-efficient DNN training. Low-precision and low-rank training reduce memory usage by exploiting numerical precision redundancy and low-dimensional structure in training tensors.

Tensor Bit-Width Reduction. Tensor bit-width reduction reduces training memory consumption by representing training tensors with fewer bits per element, as shown in Fig. 10(a). In conventional training, key tensors such as parameters, activations, gradients, and optimizer states are often stored in FP32 or FP16 formats. This approach instead stores selected tensors in lower-bit formats, such as 8-bit representations, thereby reducing the storage cost of each tensor element while preserving the original tensor shapes.

Tensor Dimensionality Reduction. Tensor dimensionality reduction reduces memory consumption by representing high-dimensional training tensors with lower-dimensional forms, as shown in Fig. 10(b). Its key idea is to avoid storing or updating full-size tensors when their main information can be captured by a much smaller representation. For example, a weight matrix $W \in \mathbb{R}^{m \times n}$ can be represented as $W \approx UV^{\top}$, where $U \in \mathbb{R}^{m \times r}$, $V \in \mathbb{R}^{n \times r}$, and $r \ll \min(m, n)$. This reduces the storage cost from mn to $(m + n)r$. Unlike LoRA [54], training-time dimensionality reduction primarily aims to reduce peak training memory rather than merely the number of parameters updated during fine-tuning.

Other Compression Methods. Beyond bit-width and dimensionality reduction, other training-time compression methods reduce memory consumption by using sparse representations or compact encodings. Sparse compression stores only nonzero values together with compact position metadata and maintains sparsity in training tensors throughout optimization. Compact encodings use masks, bitmaps, and similar lightweight representations to compress tensors during training.

Although these approaches reduce memory consumption by compressing data during training, data compression for memory optimization still faces two key issues:

- **Model Accuracy Degradation.** Compressed representations inevitably introduce information loss, causing the training trajectory to deviate from the original full-precision path and ultimately compromising model accuracy and convergence stability.
- **Training Efficiency Degradation.** Compression operations are not cost-free. They usually require additional metadata maintenance, format conversion, data reconstruction, or approximate computation, which can slow down model training.

6.1 Model Accuracy Preservation

Preserving model accuracy is the primary challenge faced by data compression during training. Since compression errors can propagate through forward outputs, backpropagation, gradient computation, and parameter updates, aggressive compression may cause the training trajectory to gradually deviate from the original full-precision one, eventually leading to accuracy loss, slower convergence, or training instability. Therefore, compression during training must carefully balance the compression ratio with model accuracy. Existing studies mainly address this challenge through **adaptive low-precision compression** and **error-controlled low-rank compression**.

6.1.1 ***Adaptive Low-Precision Compression.*** This approach is based on the observation that different training tensors have different sensitivities to numerical errors. As illustrated in Fig. 11, uniform precision compression applies the same low-precision policy to all tensors, which may excessively compress numerically sensitive tensors and cause training instability. In contrast, adaptive low-precision compression differentiates tensors based on their error sensitivity and applies varying degrees of compression. Specifically, it employs lower precision for error-insensitive tensors to minimize memory usage, while retaining higher precision for error-sensitive tensors to maintain training stability.

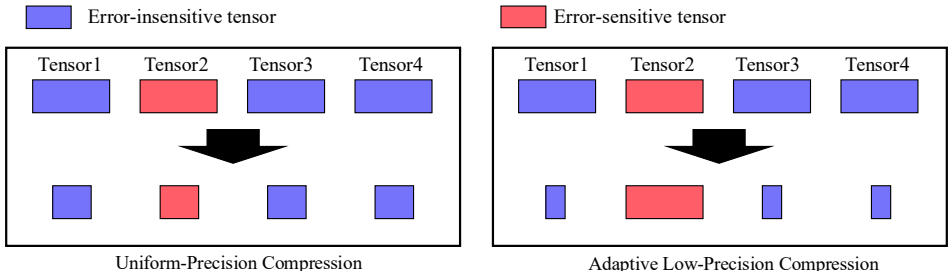


Fig. 11. Illustration of adaptive low-precision compression. Uniform-precision compression applies the same precision to all tensors, while adaptive low-precision compression assigns higher precision to error-sensitive tensors and lower precision to error-insensitive tensors.

Early low-precision training methods already show that training tensors should not be uniformly converted into a single low-precision format. Instead, they usually adopt mixed precision or adaptive precision strategies, where low precision is used for error-tolerant computations and higher precision is preserved for gradient accumulation, parameter updates, and other sensitive operations, as shown in Fig. 12. Mixed precision training [104] follows this principle by using FP16 for forward and backward computation, while retaining FP32 master weights, loss scaling, and FP32 accumulation to avoid gradient underflow and inaccurate parameter updates. However, this design still depends on additional scaling strategies. BF16 training [70] reduces this dependence by using a wider exponent range, making low-precision training less sensitive to gradient underflow.

When precision is further reduced to 8-bit, the main difficulty shifts from dynamic range alone to sensitive operators, accumulation errors, and update errors. Banner et al. [5] show that most

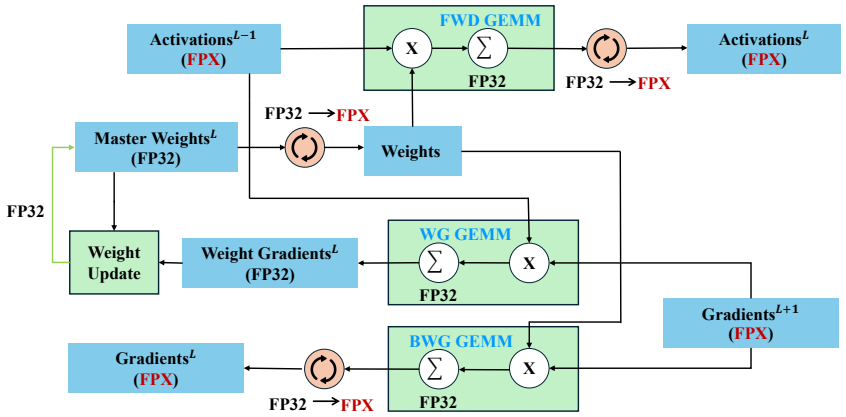


Fig. 12. Illustration of mixed precision training. Low precision is used for most forward and backward computations, while sensitive paths such as accumulation and parameter updates remain in higher precision.

training computations tolerate 8-bit representation, but batch normalization and weight-gradient updates remain precision-sensitive. They therefore replace conventional BN with range-based normalization and use different precisions for gradient propagation and weight updates. Wang et al. [165] further design training-oriented FP8 formats, chunk-based accumulation, and stochastic rounding to reduce information loss in accumulation and parameter updates.

In large-scale model training, adaptive low-precision compression extends from local operators to full training states. FP8-LM [118] observes that prior FP8 methods mainly target GEMM and leave gradients, optimizer states, and distributed communication in higher precision, limiting end-to-end memory and bandwidth savings. It therefore assigns different precisions to different training states and dynamically adjusts scaling factors to keep values within the FP8 representable range. SNIP [115] further considers layer-wise and stage-wise sensitivity. It measures the loss increase caused by forward quantization and the update deviation caused by backward quantization, and uses these signals to select layer-wise precision. Thus, low-precision training evolves from fixed-format compression toward sensitivity-guided precision allocation.

In extremely low-precision settings, numerical precision alone is insufficient to preserve training stability. FP4 training methods [24] use FP4-quantized weights in forward computation, while correcting low-bit gradient estimation errors during backpropagation and compensating for clipped activation outliers. BitNet [162] further shows that 1-bit training requires architectural adaptation rather than directly quantizing a standard Transformer. It introduces 1-bit-friendly linear layers together with activation scaling, sub-layer normalization, and group-wise quantization.

Optimizer states are also important low-precision targets because Adam-like optimizers maintain long-lived first- and second-order statistics. However, their errors can accumulate across iterations, so they cannot be blindly quantized. Dettmers et al. [33] compress Adam-like historical gradient statistics into 8-bit values using block-wise and non-uniform quantization, reducing the influence of outliers on the quantization range. Q-Adam-mini [47] further applies selective quantization: it quantizes the more robust first-order momentum, keeps sensitive second-order statistics in higher precision, and applies stochastic rounding only to embedding layers where errors tend to accumulate. COAT [173] extends low-precision training-state compression to both optimizer states and activations by reshaping optimizer-state distributions to better fit FP8 ranges and using different activation quantization granularities for different layer types.

6.1.2 ***Error-Controlled Low-Rank Compression.*** The rank of a matrix or tensor indicates how many independent dimensions are needed to represent its information. Low-rank decomposition reduces training memory by approximating high-dimensional tensors with compact factors of smaller rank. However, an overly small rank may cause large reconstruction error and weaken model expressiveness. Therefore, the key challenge is to reduce rank while preserving information essential for stable training.

Early low-rank training methods constrain weight matrices to low-rank structures during training. LRLT [138] trains weight matrices on a low-rank manifold and avoids maintaining full matrices. However, a fixed rank may be too small to preserve accuracy or too large to provide useful compression. It addresses this issue by treating training as the evolution of low-rank factors and adjusting ranks according to approximation errors. Hawkins et al. [48] extend this idea to tensorized neural networks. They use Bayesian inference to determine tensor ranks, reducing manual rank selection and avoiding rank choices that hurt accuracy or weaken compression.

For CNN training, low-rank representations also need stability from the early stage of optimization. ELRT [147] shows that replacing full-rank layers with low-rank factors may be insufficient to preserve accuracy. It selects suitable low-rank tensor formats and enforces approximate orthogonality among low-rank factors, helping low-rank CNNs maintain stable expressive capability during training. RLT [136] identifies condition-number degradation as a source of robustness loss. It observes that low-rank weights may develop unbalanced singular values when compensating for reduced parameters. To control this problem, it imposes approximate orthonormal constraints and keeps singular values within a stable range. DLRT [137] adds spectral regularization to the low-rank core matrix, controlling layer-wise condition numbers and reducing robustness degradation caused by ill-conditioned low-rank factors.

Another trend is to avoid constraining the full weight matrix to be low-rank when this may restrict the optimization trajectory. LoRITa [2] induces low-rankness during training through linear-layer composition and weight decay, followed by post-training truncation. This avoids per-iteration SVD and manual rank search. GaLore [191] shifts low-rank compression from weights to gradients. It projects gradients into a low-dimensional subspace for optimizer updates and maps the update back to the full weight space, preserving full-parameter learning. CompAct [140] compresses LLM activations rather than weights. It projects activations into a low-dimensional space, stores compressed activations, and reconstructs full-sized gradients for parameter updates. These methods show that low-rank memory optimization is moving from low-rank parameterization toward training-state compression with controlled reconstruction error.

6.2 Compression Overhead Reduction

Reducing compression overhead is crucial for preserving end-to-end training efficiency. Although compressed representations reduce memory footprint, compression introduces overhead from metadata maintenance, format conversion, reconstruction, approximate computation, and scheduling. Therefore, compression during training needs to minimize these additional costs while achieving memory savings. Existing work mainly addresses this issue through **selective compression** and **efficient compression algorithms**.

6.2.1 ***Selective Compression.*** Selective compression avoids applying the same compression policy to all tensors, layers, states, or structures. Uniform compression is simple, but it may introduce unnecessary overhead and accuracy loss. This approach therefore decides where to compress, how much to compress, and when to update the compression policy, so that compression is applied mainly where the benefit justifies the cost.

In low-precision training, SNIP [115] avoids repeated layer-wise precision trials by estimating forward loss divergence and backward update deviation, and then selecting precisions under an

efficiency budget. AC-GC [38] follows a similar idea for activation compression. It converts a tolerable loss increase into a compression constraint and determines the compression strength within a single training run.

Selective policies are also useful for activation and optimizer-state compression. ActNN [12] provides multiple optimization levels, ranging from aggressive 2-bit activation compression to near-zero-overhead modes, so users can trade memory saving for throughput overhead. Q-Adammini [47] applies selective optimizer-state quantization: it compresses the more robust first-order momentum, keeps sensitive second-order statistics in higher precision, and uses stochastic rounding only where errors tend to accumulate. These designs move compression from uniform treatment toward state- and layer-aware policies.

The same selective principle applies to sparsity-based compression methods. DSR [106] reallocates sparse parameters only at intervals rather than continuously changing the structure. RigL [39] similarly computes gradients for missing connections only at sparse update steps. By limiting when sparse structures are updated, these methods retain adaptivity without paying the search and update cost at every iteration.

6.2.2 ***Efficient Compression Algorithms.*** After the compression object and policy are selected, the training system still needs to perform compression, decompression, reconstruction, and format conversion during execution. If these operations incur substantial computational overhead, they can severely degrade overall training efficiency. Therefore, designing efficient algorithms to minimize the overhead of these operations is the most direct and effective approach. Existing works in this category can be broadly classified into two main directions:

(1) Software-Level Optimization. This approach focuses on lightweight compression and decompression operators. For compression during training, operations such as quantization, packing, unpacking, format conversion, and reconstruction may introduce extra kernel launches and memory traffic. The goal is to integrate compression into the training pipeline with minimal overhead.

Activation compression first exposes this issue. ActNN [12] uses optimized CUDA kernels to quantize activations into compact integer streams and decompress them during backward computation, reducing the runtime cost of activation compression. GACT [99] generalizes this idea to more network architectures by using runtime hooks and CUDA kernels for sensitivity estimation and activation compression. For training with ultra-low precision, Xi et al. [174] further reduce auxiliary overhead by fusing the Hadamard transform, bit splitting, and sampling into one CUDA kernel. These works demonstrate that compression efficiency depends not only on representation size but also on the implementation cost of operators related to compression.

Compact encodings reduce software overhead by integrating metadata processing into lightweight tensor operations. Gist [63] embeds mask generation and decoding into ReLU and pooling CUDA kernels, processes encoded masks directly when possible, and reconstructs tensors only before dense convolution. BitTrain [53] builds and restores bitmaps through linear scans and in-place reshaping, avoiding unnecessary copies and temporary buffers. These optimizations reduce kernel launches, memory traffic, and reconstruction overhead.

(2) Hardware-Aware Optimization. This approach focuses on whether compressed formats can be efficiently supported by physical accelerators. Computation with low bit-width may reduce arithmetic cost, but format conversion, scaling, decoding, and unsupported operations can weaken the actual speedup. Therefore, hardware-aware methods evaluate numerical format, computation units, and execution patterns jointly.

Low-precision training benefits from hardware support only when the low-precision format covers enough of the training pipeline. Early 8-bit and FP8 training methods [5, 165] show that low precision needs to be designed with accumulation, normalization, and update rules, otherwise

high-precision fallback operations may limit speedup. FP8-LM [118] extends this idea to gradients, optimizer states, and communication data in large-model training. BitNet [162] further adapts linear layers and normalization to 1-bit training, making representations with extreme low bit-width more suitable for execution with model parallelism.

Quantization granularity also affects hardware efficiency. Scaling with fine granularity can reduce numerical error, but it introduces more scaling factors and format conversions. COAT [173] balances this trade-off by using scaling at the tensor level in linear layers and scaling with finer granularity only in more sensitive nonlinear layers. ANT [44] further improves compatibility with hardware through an adaptive numerical type with fixed length. Its flint format avoids decoding data of varying lengths and memory misalignment, and can reuse existing integer MAC units with simple leading-zero detection and shifting. These works suggest that compression considering hardware constraints should evaluate bit width, scaling granularity, decoding cost, memory alignment, and compatibility with existing compute units.

SparseTrain [27] prunes activation gradients and combines sparse training dataflow with a sparsity-aware accelerator, while SWAT [128] sparsifies weights and activations so that compatible sparse hardware can reduce computation and memory traffic. Hardware-friendly dynamic sparse training [81] further constrains sparse structures to patterns that are easier to vectorize. These methods show that omitting zero values provides practical training benefits only when sparse metadata, irregular memory access, and execution patterns are efficiently supported by hardware.

7 DISTRIBUTED GPU MEMORY OPTIMIZATION

Distributed DNN training techniques harness the combined GPU memory capacity and computational resources of multiple devices, enabling the training of increasingly large-scale and sophisticated models. Existing distributed training paradigms can be broadly categorized into five types: data parallelism (DP), tensor parallelism (TP), pipeline parallelism (PP), context parallelism (CP), and expert parallelism (EP).

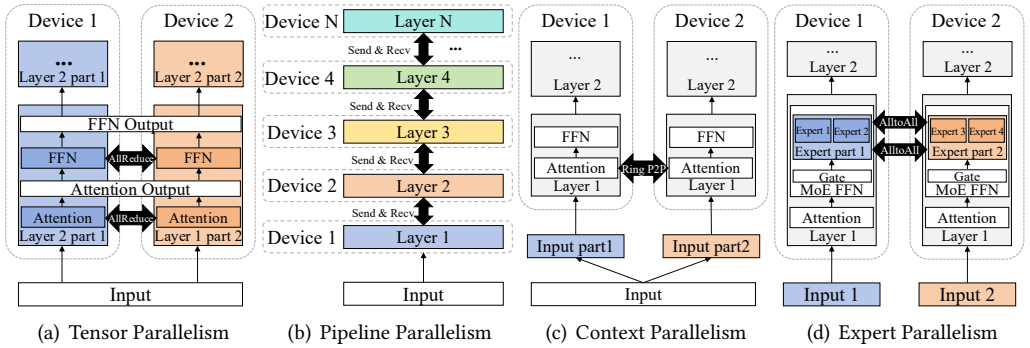


Fig. 13. Distributed training methods (Transformer examples). White and gray data represents replicas held by every device, while colorful data denotes unique data exclusive to individual devices. Notably, in EP, Input 1 and Input 2 are introduced by DP, which differs from the input partitioning in CP.

Data parallelism. DP [87] partitions the training data across multiple devices, each of which maintains a full replica of the model. Each device computes gradients based on its allocated data subset, and the gradients are subsequently aggregated through an AllReduce operation, followed by synchronous model parameter updates, as shown in Fig. 14(a).

Tensor parallelism. TP [142] partitions tensors along their row or column dimensions, so that each device stores only a slice of the tensor and performs the corresponding partial computation locally, with the final result obtained by aggregating these partial results via an AllReduce operation, as shown in Fig. 13(a).
Pipeline Parallelism. PP [58] partitions the model along the layer dimension into multiple stages, with each device storing the model data for one stage. Each device processes activations during forward propagation from the previous stage and gradients during backpropagation from the next stage, as shown in Fig. 13(b). To improve device utilization, PP divides mini-batches into fine-grained micro-batches, allowing different micro-batches to occupy different stages simultaneously.
Context Parallelism. CP [86] is tailored for training Transformer-based models in long context scenarios. It partitions input samples along the token dimension, with each device processing a subset of tokens. Since each token in the Transformer requires attention computation with all other tokens, devices perform ring-style P2P communication during the Attention module to exchange token data, as shown in Fig. 13(c).
Expert Parallelism. EP [82] is tailored for training the Mixture-of-Experts (MoE) architecture, which expands the model scale by replacing traditional FFN modules with multi-expert FFNs. EP distributes experts from the MoE FFN modules across devices, collectively forming a complete expert set. During MoE FFN computation, each device performs AlltoAll communication to exchange tokens with others and returns the processed results via another AlltoAll operation, as shown in Fig. 13(d).

Although these basic paradigms reduce memory pressure in large-scale DNN training, they still face two major challenges: inefficient device memory utilization and reduced training efficiency. These challenges appear at different system levels and can be summarized into three specific issues:

- **Memory Waste from Data Redundancy.** This challenge stems from redundant data copies across multiple devices. For example, model parameters may be replicated on each device in data-parallel training, occupying extra memory resources and reducing overall memory utilization.
- **Prolonged Residency of Activations.** This challenge stems from pipeline-style distributed training, where multiple micro-batches are processed by different devices simultaneously. Since activations must be retained until their corresponding backward computations are executed, activations from many micro-batches can stay in GPU memory for a long time, exacerbating memory pressure.
- **Memory Load Imbalance Across Devices.** This challenge stems from uneven GPU memory demands introduced by distributed parallelism. In pipeline parallelism, different stages may retain activations for different numbers of micro-batches. In expert parallelism, dynamic and potentially uneven expert routing may cause different devices to process different numbers of tokens. As a result, some devices suffer from high memory pressure while others remain underutilized, limiting the overall memory efficiency of the distributed system.

7.1 Data Redundancy Elimination

Data redundancy is prevalent across various distributed training paradigms. In DP and CP, redundant copies of model parameters, gradients, and optimizer states occupy significant GPU memory resources. In TP, devices within a TP group redundantly execute operations such as LayerNorm on all input activations, leading to redundant storage of activation tensors. Existing works primarily address this issue through **fine-grained data sharding**.

7.1.1 ***Fine-Grained Data Sharding.*** This approach partitions training data across devices, so that each device stores only its assigned shard and communicates with others to obtain missing

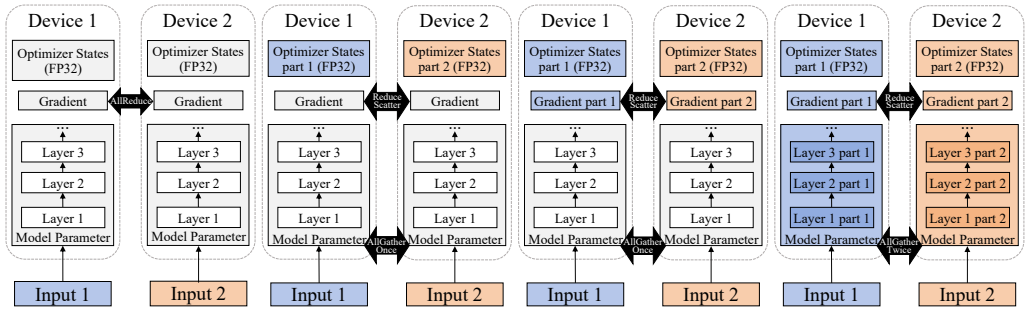


Fig. 14. Evolution of data redundancy elimination from data parallelism to ZeRO. White and Gray data represents redundant replicas held by every device, while colorful data denotes unique data exclusive to individual devices.

data when needed. By avoiding full replication of training states, data sharding reduces redundant memory usage and improves device memory utilization.

ZeRO [129] is a representative data-sharding method that reduces memory redundancy in data-parallel training through three progressive stages, as illustrated in Fig. 14. ZeRO-1 shards optimizer states across data-parallel devices, while ZeRO-2 further shards gradients. To support gradient aggregation and parameter synchronization, ZeRO-1 and ZeRO-2 replace the traditional AllReduce operation in data parallelism with ReduceScatter and AllGather. This decomposition preserves the overall communication volume while reducing the memory footprint of optimizer states and gradients.

ZeRO-3 further extends sharding to model parameters, so that each device stores only a parameter shard instead of a full model copy. This provides stronger memory savings, but it also introduces additional communication because the required parameter shards must be gathered before computation during forward and backward propagation. As the data-parallel group becomes larger, such full parameter sharding can lead to significant communication overhead.

To reduce this overhead, FSDP [195] introduces hybrid sharding, which limits sharding to a configurable subset of devices using a sharding factor F. This allows the system to trade off memory savings and communication cost: a larger sharding group saves more memory but requires more communication, while a smaller group reduces communication at the cost of higher memory usage. FSDP also performs sharding at finer granularity, materializing only the full parameters required by the current unit during forward and backward passes, thereby reducing peak runtime memory usage.

PipeMesh [85] further addresses the communication overhead of full parameter sharding by using mixed sharding. Instead of applying the same ZeRO stage to all parameters, PipeMesh assigns different sharding strategies to different parameters under a given memory budget. This parameter-level planning reduces unnecessary communication while still satisfying memory constraints.

Ulysses [62] incorporates ZeRO techniques into CP and partitions attention operations by attention heads to alleviate memory pressure during long sequence training. It replaces the traditional ring-style P2P communication with AlltoAll operations to efficiently exchange data required for attention head partitioning. Megatron [76] introduces a token partitioning strategy similar to that used in CP to reduce activation redundancy in TP, where each TP device only computes LayerNorm, Dropout, and similar operators for the tokens it is responsible for, eliminating redundant token

computations and activations. To support attention with token partitioning, Megatron decomposes the original AllReduce into an AllGather followed by a ReduceScatter, without increasing the overall communication volume.

7.2 Prolonged Activation Memory Minimization

During DNN training, backward propagation depends on activations from the forward pass for gradient computation, which must be retained in GPU memory until their corresponding backward steps are completed. This dependency also holds in distributed training. However, in pipeline parallelism, multiple micro-batches are processed concurrently across stages, while the forward and backward phases are executed sequentially, causing activations to remain in GPU memory for much longer. For example, GPipe [58] schedules micro-batches by executing all forward passes prior to any backward computation. This rigid phase separation causes activations to remain in GPU memory for extended periods, resulting in inefficient memory utilization, as illustrated in Fig. 15(a). To address this issue, existing works follow two approaches: **(1) scheduling strategy optimization** and **(2) integration of memory optimizations**.

7.2.1 ***Scheduling Strategy Optimization.*** Such methods reduce GPU memory footprint by adjusting the execution order of forward and backward passes in the pipeline, enabling the early release of activation values. As a pioneering work in optimizing pipeline scheduling strategies, PipeDream [107] proposes the One-Forward-One-Backward (1F1B) scheduling strategy, where each device alternates between forward and backward computation, as shown in Fig. 15(b). With this scheduling mechanism, regardless of the number of micro-batches injected per batch, the peak activation memory footprint per device during the steady state approximates the memory footprint of a single micro-batch. Furthermore, PipeDream removes pipeline flushing and adopts an asynchronous pipeline mechanism to reduce the bubble rate. However, such asynchronous updates require maintaining multiple parameter versions, which not only increases GPU memory overhead but also poses risks to model performance. To address this, PipeDream-2BW [108] introduces double-buffered weight updates (2BW), which accumulate gradients from multiple micro-batches and maintain only two versions of model parameters used for forward and backward passes, thus reducing the high GPU memory overhead of PipeDream's multi-version scheme.

While PipeDream-2BW reduces the memory overhead of multi-version parameters, it still follows an asynchronous update paradigm. In contrast, DAPPLE [40] presents a synchronous pipeline variant of the 1F1B strategy, as shown in Fig. 15(c). It periodically flushes the pipeline to synchronize parameter updates, thereby avoiding the need to maintain multiple parameter versions and improving convergence stability. However, pipeline flushing increases the bubble rate and reduces pipeline utilization.

To reduce these bubbles, Narayanan et al. [110] propose an interleaved pipeline schedule. It partitions the model into finer-grained chunks and assigns multiple non-contiguous chunks to each pipeline stage. This allows the pipeline to be filled earlier, reducing idle bubbles while preserving memory efficiency. Another line of work reduces pipeline bubbles through token-level partitioning, pioneered by TeraPipe [89]. Instead of treating each input sequence as an indivisible unit, token-level partitioning splits the sequence along the token dimension, allowing downstream stages to start processing partial token streams earlier. Building on this idea, MEPipe [149] combines token partitioning with interleaved scheduling to support flexible pipeline configurations under different memory budgets. SlimPipe [88] further uses token-level splitting to start the 1F1B pipeline with only one micro-batch, reducing peak activation memory per device and improving scalability for long-sequence training.

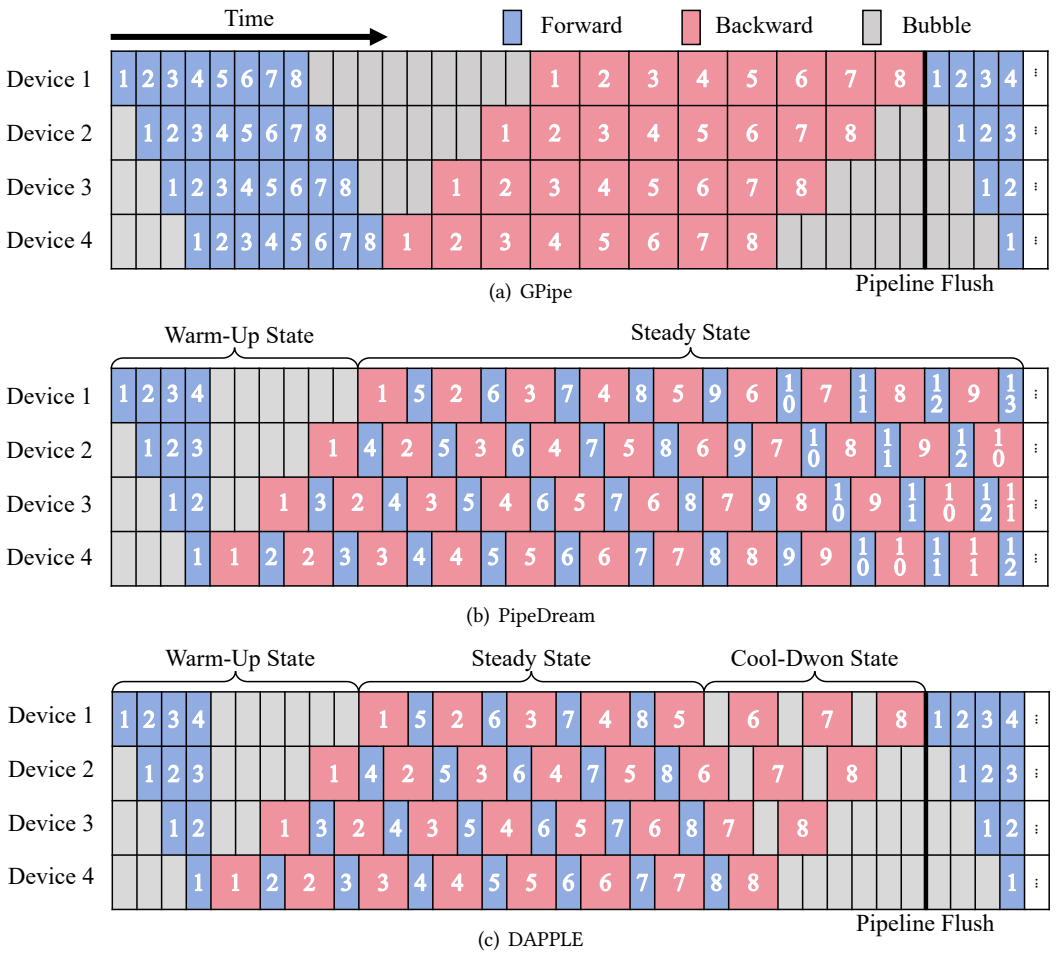
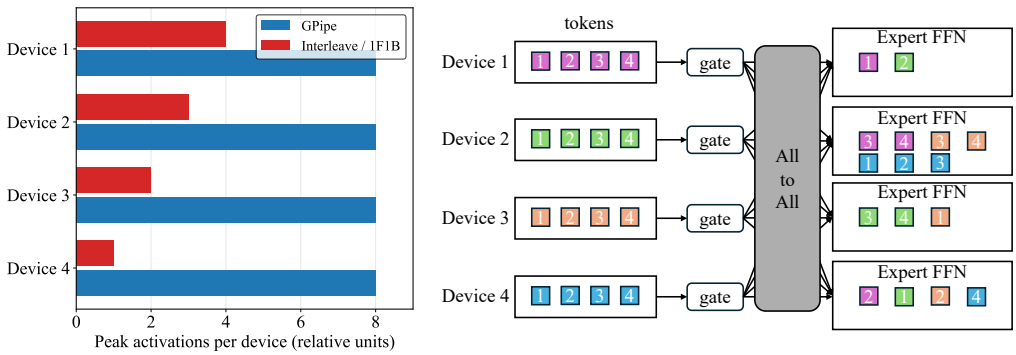


Fig. 15. Comparison of pipeline parallel scheduling strategies. To simplify the illustration, the backward propagation time is assumed to be twice the forward propagation time.

7.2.2 ***Integration of Memory Optimizations.*** Such methods combine memory optimization techniques like recomputation and offloading to address the prolonged residency of activations. In fact, GPipe [58] already employs recomputation within individual pipeline stages to reduce GPU memory usage. PipeMare [177] further refines this approach by partitioning the model into several segments (each spanning multiple pipeline stages) and retaining activation checkpoints only at segment boundaries for recomputation, thereby significantly reducing the GPU memory footprint of activations in fine-grained pipeline parallelism scenarios. PipeOffload [160] introduces a selective activation offloading strategy that prioritizes offloading activations with longer lifespans, specifically those with extended intervals between forward and backward passes. SPPO [18] proposes an adaptive sequence pipeline parallel offloading strategy that uses two-level activation management to keep frequently accessed skeletal activations in GPU memory while dynamically adjusting the offloading ratio to maximize computation-offloading overlap.

7.3 Memory Load Balance Across Devices

Load imbalance is a common issue in distributed systems, and GPU memory load imbalance is equally prevalent in distributed training scenarios. In PP with the 1F1B scheduling strategy [40, 107, 108, 110], devices with smaller PP ranks store more activations due to the later backward pass, leading to imbalanced GPU memory usage across stages. Without using 1F1B, as in GPipe, while load balancing is achieved, GPU memory efficiency is significantly reduced, as shown in Fig. 16(a). In EP [46, 82, 124, 141], the random routing of tokens to experts can lead to imbalanced expert workloads, causing disparities in both computational and GPU memory loads across devices, as shown in Fig. 16(b). In hybrid parallelism training scenarios that employ multiple parallel paradigms such as DP, TP, PP, CP and EP, most existing auto-parallelism works [94, 103, 151, 196] treat GPU memory primarily as a feasibility constraint and optimize only for training throughput, which can still lead to imbalanced GPU memory load across devices. Since different devices may carry uneven memory loads during distributed training, some GPUs can become memory bottlenecks while others still have unused memory resources. This imbalance reduces overall GPU memory efficiency and limits the scale of models that can be trained [73, 84, 148, 193]. To address this issue, existing works primarily tackle the problem through four strategies: **(1) load redistribution**, **(2) combination with memory optimization**, **(3) algorithmic optimization**, and **(4) memory-aware parallel orchestration**.



(a) Memory Load Imbalance in 1F1B Scheduling (b) Imbalanced Expert Workloads in EP

Fig. 16. Memory load imbalance in distributed training. (a) Memory load imbalance in 1F1B scheduling, where earlier pipeline stages have higher memory loads; (b) Memory load imbalance in EP caused by the random routing of tokens by the gating function.

7.3.1 ***Load Redistribution.*** To improve GPU memory utilization during DNN training, these methods rebalance memory loads across devices through dynamic pipeline repartitioning and inter-device activation migration. By reducing memory load imbalance, they better utilize available GPU memory and support larger model training.

In essence, these methods improve overall GPU memory utilization by shifting part of the memory pressure from heavily loaded devices to lightly loaded ones. Since this redistribution requires extra data movement between devices, a key design goal is to balance memory loads while keeping communication overhead low. vPIPE [193] proposes an online multi-level graph partitioning strategy that leverages real-time runtime signals from each stage to dynamically adjust layer assignments during training. To prevent layer redistribution from stalling the training process,

it further designs a live layer migration protocol, where asynchronous data migrations are executed in parallel with ongoing training. BPipe [73] directionally transfers activations between paired pipeline stages and introduces a scheduling algorithm to minimize migration costs while ensuring computation continuity. Leveraging NVLink's high-speed bandwidth, MPress [197] develops an efficient D2D tensor swapping technique that exclusively migrates short-lived tensors, accounting for the limited spare GPU memory. Targeting long sequence training scenarios, SlimPipe [88] proposes an attention context exchange mechanism. Devices with heavy loads offload partial Q and KV data to underutilized devices. After completing part of the attention computation, the underutilized devices return the partial attention output, effectively balancing both GPU memory usage and computational load across devices.

7.3.2 ***Combination with Memory Optimization.*** This category of methods accounts for activation memory pressure across different stages and achieves memory load balancing primarily by adopting varying degrees of offloading and recomputation strategies tailored to each stage. vPIPE [193] leverages a Virtualized Tensor Manager to dynamically track tensor storage states. By calculating the ratio of memory savings benefits to offloading and recomputation costs for each layer, it prioritizes asynchronous offloading and only performs selective recomputation on layers with high offloading costs. MPress [197] replaces inter-device activation migration with recomputation only when D2D swap is unavailable (i.e., when GPU free memory is extremely limited) and tensor lifetimes are extremely short.

While methods like vPIPE allow dynamic adjustment of recomputation strategies for distinct pipeline stages, they only support layer-level granularity when determining whether to conduct recomputation, which fails to finely regulate the activation memory footprint across individual stages and thus yields suboptimal performance in scenarios with a large number of pipeline stages. Merak [80] mitigates this limitation by proposing stage-aware recomputation that refines the recomputation granularity to independent functional modules such as attention blocks and FFN blocks while balancing memory pressure across stages by dynamically tuning the non-recomputation ratio for each pipeline stage. AdaPipe [148] further proposes fine-grained recomputation at the intra-layer operator level. By profiling the memory and computational overheads of individual operators, it customizes recomputation plans for individual pipeline stages.

7.3.3 ***Algorithmic Optimization.*** This category addresses memory load imbalance in MoE architectures caused by uneven expert assignment. These methods improve expert load distribution through algorithmic designs, thereby balancing GPU memory usage across devices. They can be divided into two types:

(1) Auxiliary Loss-driven Balancing. This method introduces an auxiliary loss related to expert load balancing and incorporates it into the total loss, which can be formulated as:

$$L_{\text{total}} = L_{\text{task}} + \alpha \cdot L_{\text{balance}} \tag{12}$$

where L_{task} is the primary task loss, α is the balancing coefficient, and L_{balance} is the auxiliary load balancing loss (LBL). Shazeer et al. [141] and GShard [82] compute the LBL using the proportion of tokens processed by each expert and the gating weights assigned to experts for token routing. Qiu et al. [124] point out that LBL is typically computed based on partial data within the mini-batch, which tends to make the model overemphasize expert load balancing and thus suppress expert specialization. To mitigate this issue, they extend the scope of LBL computation to the entire global batch. To better reconcile load balancing with expert specialization, Guo et al. [46] further augment the original LBL with two additional terms: an orthogonality loss (L_o) designed to strengthen expert specialization and a variance loss (L_v) to enhance routing diversification, which alleviates the overall MoE performance degradation while maintaining effective load balancing.

(2) Loss-Free Direct Balancing. This method avoids introducing extra loss terms and instead achieves load balancing by directly modifying the routing mechanism. GShard [82] and Switch Transformers [41] propose the expert capacity, where each expert is assigned a fixed threshold for the number of tokens it can process and prohibited from exceeding this limit. If the routed expert for a token has already reached full capacity, the token's computation on that expert is discarded to mitigate memory overflow caused by expert overload. BASE [83] argues that dropping tokens under expert capacity constraints can harm model quality. To avoid unprocessed tokens, BASE formulates token allocation as a linear assignment problem with constraints where it enforces exactly equal numbers of tokens per expert while maximizing token-expert affinities. Wang et al. [163] add an expert bias to routing scores before making Top-K routing decisions. The bias is updated dynamically based on each expert's recent load, encouraging a more balanced allocation across experts in subsequent routing steps.

7.3.4 ***Memory-Aware Parallel Orchestration.*** These methods address memory load imbalance by optimizing parallelism degrees, model partitioning, and expert placement in distributed training. Early efforts mainly focus on balancing memory usage in pipeline parallelism. mCAP [36] uses incremental profiling to capture the memory footprint of each model layer and builds a predictive model to estimate the memory consumption of different pipeline partitioning schemes, with the objective of minimizing the maximum predicted peak memory across GPUs. Similarly, AdaPipe [148] performs layer-level model partitioning to balance memory loads, while DawnPiper [120] further refines this direction by modeling the computation graph at a finer granularity.

Beyond pipeline partitioning, another line of work incorporates memory balance into automatic parallelism search. HIPPIE [84] introduces the memory efficiency as an optimization metric to capture the trade-off among GPU memory footprint, throughput, and scalability, and uses it to derive balanced hybrid-parallel configurations. Galvatron-BMW [167] further defines *Memory Balance Degree* (α_m) and *Computation Balance Degree* (α_t). It prunes the auto-parallel search space across multiple parallelism dimensions using a decision-tree-based procedure, and then applies a heuristic algorithm to identify configurations that jointly improve memory and computation balance. SmartMoE [186] extends automatic parallelization to MoE training, where expert workloads vary across layers and iterations. It constructs a pool of candidate execution plans offline and dynamically selects suitable expert placements at runtime, thereby redistributing expert-related memory and computation pressure across devices.

For MoE models, memory imbalance can also arise at runtime due to skewed expert selection, where popular experts overload their hosting devices while other devices host fewer or less active experts and retain spare memory capacity. FasterMoE [49] mitigates this problem through dynamic shadowing, which uses this spare capacity to replicate popular experts on lightly loaded devices, allowing part of their token computation to be executed locally. This reduces pressure on the original expert-hosting devices and improves overall hardware utilization.

8 OTHER OPTIMIZATIONS

Beyond the four mainstream approaches discussed earlier, this section surveys a set of complementary techniques that target runtime memory management in GPU training systems. Unlike previous categories, which mainly reduce memory pressure by offloading, recomputing, compressing, or partitioning training tensors, these techniques focus on reducing allocator-induced fragmentation, improving memory reuse, and minimizing temporary buffers. They are orthogonal to the mainstream methods and can be used together with them.

8.1 GPU Memory Defragmentation

Frequent allocation and deallocation of memory blocks with varying sizes and lifetimes can divide free memory into small non-contiguous blocks, leading to memory fragmentation [28, 29]. During DNN training, the native CUDA allocation and deallocation primitives, `cudaMalloc` and `cudaFree`, incur significant overhead due to mandatory CPU-GPU synchronization, which can severely degrade training throughput. Consequently, modern deep learning frameworks such as PyTorch [117] and TensorFlow [1] adopt memory pool-based GPU memory allocators that reuse previously reserved device memory and reduce interactions with the CUDA runtime. In PyTorch, allocation requests are handled by a best-fit [6] strategy that splits cached blocks in the memory pool to satisfy requested sizes, invoking `cudaMalloc` only when the pool is exhausted. For deallocation, the allocator simply marks the corresponding block as inactive and attempts to coalesce adjacent free blocks within the pool, without returning memory to the CUDA runtime via `cudaFree`. However, the frequent alternation between GPU memory allocation and deallocation in DNN training progressively fragments the memory pool into many non-contiguous free blocks, thereby degrading effective memory utilization. As shown in Fig. 17, after multiple allocation and deallocation operations, even though the pool contains sufficient aggregate free space to satisfy a 1 GB request, the absence of a contiguous free block prevents the PyTorch caching allocator from serving the request from the pool. Consequently, a new memory block must be requested from the CUDA runtime via `cudaMalloc`.

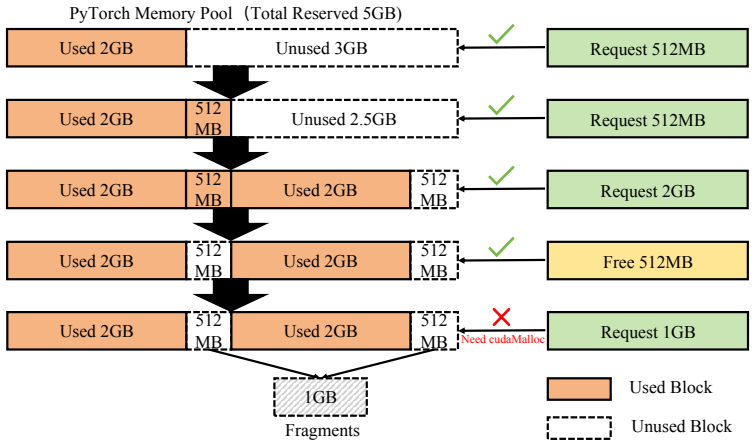


Fig. 17. GPU memory fragmentation in PyTorch's caching allocator.

With the continuous scaling of model sizes, the increasing adoption of memory optimization techniques such as offloading and recomputation has further amplified the frequency of GPU memory allocation and deallocation, exacerbating GPU memory fragmentation and thereby degrading both GPU memory utilization efficiency and training performance [45, 61]. To tackle this issue, prior work has proposed a variety of approaches from different system design perspectives, which can be broadly categorized into three classes: **(1) static partitioning**, **(2) preplanned allocation**, and **(3) runtime fragmentation management**.

8.1.1 ***Static Partitioning.*** Static partitioning exploits the distinct lifetime characteristics of tensors during DNN training. Certain tensors persist throughout an entire training iteration, such as model parameters and optimizer states, whereas others are short-lived and exist only during localized computation phases, including activation tensors generated under recomputation

or offloading. Since memory fragmentation is primarily induced by the frequent allocation and deallocation of short-lived tensors, static partitioning divides the memory pool into dedicated regions for long-lived and short-lived tensors so that fragmentation is confined to the short-lived region, eliminating its impact on memory used by long-lived tensors.

vDNN++ [143] optimizes GPU memory fragmentation for offloading techniques by predicting the required memory pool capacity for training, and allocates offloaded tensors and non-offloaded tensors to the high-address and low-address regions of the memory pool respectively. Since offloaded tensors are freed each iteration, this region remains contiguously available. DyMem [178] accounts for the dynamic dependencies in nonlinear DNNs [50] by classifying training tensors into three categories based on their mobility and further partitions the memory pool into a main space and an incremental space, with the main space subdivided into low-address and high-address regions. Coop [187] addresses fragmentation caused by non-contiguous memory releases under recomputation by using a sliding window algorithm that selects tensors from contiguous address ranges for recomputation and places tensors in different pool regions based on recomputation cost, prioritizing low-cost tensors to reduce both fragmentation and recomputation overhead.

8.1.2 ***Preplanned Allocation.*** While static partitioning isolates different classes of tensors into coarse-grained memory regions, it does not fully specify how individual tensors should share and reuse memory inside each region. Preplanned allocation further performs tensor-level planning: it assigns tensors to concrete memory addresses according to their lifetimes and reuse opportunities before execution, thereby reducing fragmentation caused by runtime allocation decisions.

Preplanned allocation methods leverage the largely invariant tensor lifetimes and allocation order across DNN training iterations by formulating GPU memory management as a Dynamic Storage Allocation (DSA) problem and solving near-optimal allocation plans offline via profiling or static analysis. The resulting plan is enforced at runtime, eliminating fragmentation caused by dynamic allocation decisions.

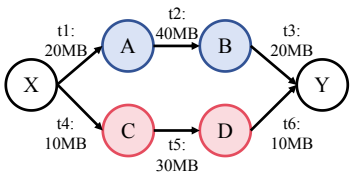
MegTaiChi [55] profiles invariant tensor access patterns in the dynamic computational graph (DCG) during the first five training iterations and then applies memory allocation simulation with six efficient memory management rules to derive a near-optimal tensor memory allocation plan prior to training. MODeL [146] models tensor memory allocation as an ILP problem derived from the DNN computation graph and solves it using off-the-shelf ILP solvers to obtain optimal GPU memory allocation plans. Imanishi et al. [61] transform the Offline DSA problem into a topological ordering optimization problem for memory allocation. With the constraints of topological ordering inherently ensuring memory safety, they employ the simulated annealing algorithm to search for a near-optimal topological ordering, thereby achieving a low-fragmentation GPU memory allocation strategy. STAlloc [60] further observes pronounced spatio-temporal regularities in tensor allocation during large-scale model training. Spatial regularity arises from the repetitive structure of Transformer layers, leading to highly repetitive tensor sizes, while temporal regularity stems from fixed tensor lifetime patterns. To exploit these properties, STAlloc hierarchically groups tensors by first forming HomoPhase Groups with similar lifetimes and then subdividing them into HomoSize Groups with identical sizes, enabling fine-grained intra-group allocation and inter-group memory reuse to derive a static allocation plan.

8.1.3 ***Runtime Fragmentation Management.*** Methods in this category dynamically adjust GPU memory allocation strategies based on real-time GPU memory fragmentation status. Building on CUDA's low-level Virtual Memory Management (VMM) primitives [121], GMLake [45] designs and implements a GPU memory allocator with fragmentation stitching capability, whose core resides in the Virtual Memory Stitching mechanism. When the memory pool lacks a single contiguous block large enough to satisfy an allocation request, but the aggregate free capacity across fragmented

blocks is adequate, the GMLake allocator leverages virtual memory address mapping to stitch multiple physically discontiguous memory blocks into a virtually contiguous address range, thereby satisfying the allocation request and substantially alleviating memory fragmentation. In contrast, T-Control [169] proposes Segment-centered Memory Management, which prioritizes allocating GPU memory requests to segments with high occupancy and dynamically migrates memory blocks to maintain the contiguity of GPU memory within segments. This approach reduces the degree of fragmentation and improves GPU memory utilization efficiency.

8.2 Temporary Buffer Reduction

Deep learning frameworks typically construct a computational graph from the model definition, where nodes correspond to operators and edges capture tensor data dependencies. During GPU execution, operators often require additional temporary buffers to hold intermediate results, which are released once the operator completes. While these temporary buffers have relatively short lifetimes, their allocation and deallocation are tightly constrained by operator semantics, implementation strategies, and data dependencies, preventing flexible adjustment of their live ranges at runtime. Consequently, temporary buffers may occupy a large portion of GPU memory, substantially reducing overall memory utilization efficiency. As shown in Fig. 18, within a fused operator, different internal operators impose markedly different demands on temporary buffers. Variations in the execution order directly affect the allocation and deallocation timing of these buffers, resulting in different overlaps of their live ranges and thus significantly different memory peaks (e.g., decreasing from 90 MB to 70 MB).



Operator order 1	Temporary buffer	Peak Mem: 90MB
A	t1 t2 t4	70MB
C	t4 t5 t2	80MB
B	t2 t3 t5	90MB
D	t5 t6 t3	60MB

Operator order 2	temporary buffer	Peak Mem: 70MB
A	t1 t2 t4	70MB
B	t2 t3 t4	70MB
C	t4 t5 t3	60MB
D	t5 t6 t3	60MB

Fig. 18. Impact of operator execution order on temporary buffer and GPU memory peak.

Motivated by the memory overhead introduced by temporary buffers, existing work has proposed optimization techniques at both the operator implementation level and the computation graph level. Accordingly, we organize related approaches into two categories: **(1) operator optimization** and **(2) computation graph optimization**.

8.2.1 ***Operator Optimization.*** Operator optimization focuses on improving the implementation of individual operators or fused operators by avoiding the explicit materialization of memory-intensive intermediate results within operators, thereby reducing the memory footprint of temporary buffers.

In conventional convolution operators, the widely adopted im2col algorithm explicitly materializes an intermediate matrix with substantial data redundancy. To address this issue, MEC [25] proposes an improved approach based on compact lowering, which reduces redundancy in the intermediate matrix by batching the unfolding of input submatrices. For large-scale pair-wise interaction operators (e.g., Gaussian kernel matrix-vector products), KeOps [11] leverages symbolic tensor techniques to tightly fuse element-wise computations with subsequent reduction operations, thereby avoiding the explicit storage of memory-intensive kernel matrices and reducing memory consumption from quadratic complexity to linear complexity.

In Transformer architectures, the Attention operator constitutes a major source of memory pressure because it explicitly materializes a quadratic-complexity attention matrix. FlashAttention [31] mitigates this overhead by employing tiling and recomputation to fuse matrix multiplication, softmax, and subsequent operations into a single kernel, and by using incremental softmax to ensure numerical correctness. Throughout the computation, only linear-complexity intermediate statistics are maintained, significantly reducing the memory requirements of temporary buffers. FlashAttention-2 [30] and FlashAttention-3 [139] further exploit GPU hardware characteristics, such as warp-level work partitioning, to improve hardware utilization and computational efficiency. Building on FlashAttention, Blockwise Parallel Transformers (BPT) [95] extend blockwise computation to the FFN modules and further fuse the execution of Attention and FFN. As a result, BPT reduces the GPU memory consumption of FFN intermediate activations by up to four times compared to FlashAttention, substantially improving memory efficiency when processing long sequences. Liu et al. [97] further combine BPT with CP for distributed training, enabling the model to scale to even longer context lengths.

8.2.2 ***Computation Graph Optimization.*** While operator optimizations effectively reduce temporary buffer usage within individual or locally fused operators, memory peaks in complex models are largely determined by operator scheduling and tensor lifetimes at the computation graph level. Consequently, computation graph optimization reduces the overall memory peak by jointly adjusting graph structure and operator execution order to shorten the live ranges of intermediate tensors and temporary buffers.

A straightforward yet effective strategy is to reschedule the operator execution order to enable direct buffer reuse. LayRub [68] observes that the standard backward pass computes activation gradients first, which requires retaining both activations and their gradients simultaneously. By prioritizing weight gradients, it enables the early release of activation memory, allowing subsequent activation gradient computations to directly reuse the freed space. Similarly, MPMoE [190] splits the AlltoAll communication and MoE FFN computation into micro-batches for pipelined processing, and proactively designs a GPU memory buffer reuse mechanism that enables homogeneous tensors from different micro-batches to share the same GPU memory space temporally.

Beyond these explicit buffer reuse strategies, advanced graph-level optimization frameworks further optimize memory by jointly restructuring the computation graph and refining the execution order. eXLA [3] provides a user-transparent computation graph memory optimization compiler extension that leverages match-and-replace transformations to automatically substitute operators with more memory-efficient implementations. It further applies operator reordering and data-flow graph splitting to execute large tensors as sequential slices, reducing memory usage by enabling timely release of temporary workspace. MODeL [146] simultaneously optimizes the tensor memory allocation scheme while refining the operator scheduling order. Considering the high cost of full computation graph enumeration, ROAM [144] divides the graph into independent subgraphs and dependent subgraphs, optimizes operator execution order within each subgraph, and facilitates inter-subgraph memory reuse, ultimately lowering the peak memory footprint during training. Nevertheless, existing approaches generally rely on the assumption of statically known tensor shapes, which limits their applicability to dynamic shape scenarios. Relax [79] tackles this challenge by introducing a unified cross-level abstraction spanning computation graphs, operator kernels, and external libraries, together with first-class symbolic shape annotations that enable dynamic shape–aware memory planning and operator fusion, thereby reducing memory peaks in dynamic shape workloads.

9 DISCUSSION ON FUTURE TRENDS

The GPU memory optimization techniques discussed above have significantly alleviated the GPU memory bottleneck in current DNN training systems. However, memory optimization remains an evolving problem as models, hardware, and training clusters continue to change. From the software side, emerging model architectures introduce new memory behaviors and more diverse memory demands. From the hardware side, new memory and interconnect technologies create opportunities to optimize data placement across broader memory hierarchies. From the system side, evolving GPU cluster architectures introduce new challenges for cross-device memory coordination. Therefore, this section discusses future challenges and opportunities from these three perspectives and summarizes several emerging development trends.

Evolution of Model Architectures. The evolution of modern models is reflected not only in their increasing parameter scale, but also in the continuous change of model architectures. Recent models are moving from static, dense, and homogeneous architectures toward more dynamic, sparse, and modular architectures. This architectural evolution introduces new memory behaviors that are difficult to handle with conventional memory optimization techniques.

First, advanced MoE models [82, 93] introduce dynamic expert activation, where the executed experts and the corresponding memory demand may vary across inputs, layers, and training iterations. This dynamic behavior challenges memory optimization strategies that rely on static execution graphs, predictable tensor lifetimes, or stable peak memory patterns. Although Section 7.3.3 has discussed existing MoE-specific optimization methods, dynamic expert activation remains difficult for general-purpose memory optimization techniques that assume stable execution and memory patterns.

Second, emerging attention architectures, such as Linear Attention [72, 176] and Sparse Attention [23, 184], are increasingly adopted to improve the efficiency of long-sequence modeling. These architectures change the computation and memory characteristics of key model components, making it necessary to revisit memory optimization methods originally designed for dense attention or standard Transformer blocks.

Third, multimodal models [96, 125] further increase architectural heterogeneity. They are usually composed of multiple structurally different modules, such as encoders, backbones, projectors, and decoders, which exhibit distinct memory footprints and computation patterns. Moreover, multimodal training often involves mechanisms such as stage-wise training and parameter freezing [153], which make the execution process more complex and reduce the effectiveness of a single unified memory optimization strategy.

Therefore, future memory optimization techniques need to better adapt to emerging model architectures. Instead of assuming static and homogeneous model structures, they should develop more architecture-aware optimization strategies that can handle dynamic execution, efficient attention mechanisms, and heterogeneous module composition.

Emerging Opportunities from New Hardware Technologies. The development of emerging hardware technologies is expanding the design space of memory optimization. In particular, new memory interconnects and heterogeneous memory devices are reducing the cost of moving data across memory tiers, creating opportunities to rethink traditional GPU-centric memory management.

Compute Express Link (CXL) [26] enables cache coherence and fine-grained data migration between processors and memory devices, while NVLink-C2C [114] provides high-bandwidth CPU-GPU interconnection. These hardware capabilities substantially reduce CPU-GPU data migration costs, making offloading-based techniques more practical and efficient. Recent systems such as TECO [175] and SuperOffload [90] have already begun to exploit CXL and NVLink-C2C to design

more efficient offloading schemes. These efforts indicate that emerging interconnect technologies can make external memory a more practical extension of GPU memory for large-scale DNN training.

In addition, novel architectures such as Processing-in-Memory (PIM) [22, 145, 172, 181] enable computation to be performed closer to where data reside. When integrated into the existing memory and storage stack, PIM opens new opportunities for training optimization by supporting both memory expansion and near-data execution. For example, some model parameters or optimizer states can be offloaded to PIM-enabled memory devices, while memory-intensive operations can be executed near the offloaded data. This reduces GPU memory consumption and cross-tier data migration, while improving the efficiency of large-scale DNN training.

As these hardware technologies mature and become more widely available, future memory optimization techniques may move beyond treating GPU memory as the only high-performance memory resource. Instead, they can coordinate GPU memory, host memory, CXL-attached memory, and storage devices in a more unified manner, thereby further alleviating the GPU memory wall in DNN training.

Evolution of GPU Cluster Architectures. GPU cluster architectures are also becoming increasingly complex, with the emergence of heterogeneous GPU clusters and tightly interconnected supernode architectures. These environments introduce more diverse device capabilities, memory capacities, and interconnect characteristics, creating new challenges and opportunities for GPU memory optimization.

First, heterogeneous distributed training [35, 116, 157] has attracted increasing attention as GPU hardware rapidly evolves and computing resources accumulate over time. In such environments, devices from different generations or vendors may coexist in the same training system. However, their computation capability and memory capacity are not always well aligned [109]. Therefore, directly applying memory optimization strategies designed for uniform GPU clusters may lead to uneven memory utilization and limit overall system efficiency. Existing studies on heterogeneous training [157, 196] mainly focus on improving training throughput, while memory optimization for heterogeneous devices remains relatively underexplored.

Second, supernode architectures, such as Huawei CloudMatrix384 [200] and NVL72 [111], are emerging to reduce cross-node communication bottlenecks in large-scale training. These architectures connect a large number of GPUs with high-bandwidth and low-latency interconnects, weakening the traditional boundary between devices and nodes. However, many existing memory optimization techniques are designed to avoid inter-device data movement due to communication overhead. In supernode architectures, high-bandwidth interconnects significantly reduce the cost of data movement across GPUs, creating new opportunities to trade communication for memory capacity. Future methods can exploit this property by dynamically placing, migrating, or sharing tensors across tightly connected GPUs to improve overall memory utilization.

Overall, the evolution of GPU cluster architectures introduces new decision dimensions for memory optimization. Future methods need to move beyond device- or node-local optimization and adopt a more global system-level design that jointly considers computation capability, memory capacity, interconnect bandwidth, and resource heterogeneity.

Given the aforementioned challenges and opportunities, future GPU memory optimization is expected to evolve in the following directions:

- **Toward Architecture-Aware Memory Optimization.** Emerging model architectures introduce new memory behaviors that differ from conventional dense Transformer models. For example, MoE models bring input-dependent expert activation, efficient attention mechanisms change the memory and computation patterns of attention modules, and multimodal models combine heterogeneous components with different memory footprints. Therefore,

future memory optimization should become more architecture-aware, adapting its strategies to the execution patterns and memory characteristics of different model structures.

- **Toward Hierarchy-Aware Memory Optimization.** Emerging hardware technologies are expanding the available memory hierarchy beyond GPU memory alone. High-bandwidth CPU-GPU interconnects, CXL-attached memory, storage-class memory, and PIM-enabled devices provide new opportunities for using external memory resources more efficiently. Therefore, future memory optimization should better coordinate data placement, data movement, and computation placement across heterogeneous memory and storage tiers, rather than treating GPU memory as the only effective optimization target.
- **Toward Topology-Aware Global Memory Coordination.** Evolving GPU cluster architectures, such as heterogeneous GPU clusters and tightly interconnected supernodes, introduce more complex device capabilities, memory capacities, and interconnect characteristics. Future memory optimization should move beyond device-local or node-local decisions and coordinate memory usage across the entire training system. By jointly considering memory capacity, computation capability, and interconnect bandwidth, future methods can better balance memory pressure and exploit high-bandwidth links for tensor placement, migration, and sharing.

10 CONCLUSION

In this survey, we provide a comprehensive analysis of GPU memory optimization for DNN training, which has become increasingly important due to the rapid growth of model size and training data volume. We explore the main sources of GPU memory consumption and the fundamental challenges they introduce in DNN training scenarios. We then review existing memory optimization techniques, including data offloading, activation recomputation, data compression, distributed training, and other complementary optimizations.

Our analysis covers the core principles of these methods and provides a taxonomy based on their optimization mechanisms. We further investigate representative approaches and discuss their benefits in improving memory efficiency, along with the system overhead they incur. Finally, we discuss emerging trends in GPU memory optimization for DNN training, providing useful guidance for future systems research in this area and for the development of more efficient DNN training techniques.

REFERENCES

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. TensorFlow: A System for Large-Scale Machine Learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation*.

[2] Ismail R. Alkhouri, Xitong Zhang, and Rongrong Wang. 2024. Structure-Preserving Network Compression via Low-Rank Induced Training through Linear Layers Composition. *Transactions on Machine Learning Research* (2024).

[3] Artem Artemev, Yuze An, Tilman Roeder, and Mark van der Wilk. 2022. Memory Safe Computations with XLA Compiler. In *Proceedings of the 35th Advances in Neural Information Processing Systems*.

[4] Jonghyun Bae, Jongsung Lee, Yunho Jin, Sam Son, Shine Kim, Hakbeom Jang, Tae Jun Ham, and Jae W Lee. 2021. FlashNeuron: SSD-Enabled Large-Batch Training of Very Deep Neural Networks. In *Proceedings of the 19th USENIX Conference on File and Storage Technologies*.

[5] Ron Banner, Itay Hubara, Elad Hoffer, and Daniel Soudry. 2018. Scalable Methods for 8-Bit Training of Neural Networks. In *Proceedings of the 32nd Advances in Neural Information Processing Systems*.

[6] Carter Bays. 1977. A Comparison of Next-Fit, First-Fit, and Best-Fit. *Commun. ACM* (1977).

[7] Olivier Beaumont, Lionel Eyraud-Dubois, Julien Herrmann, Alexis Joly, and Alena Shilova. 2024. Optimal Re-Materialization Strategies for Heterogeneous Chains: How to Train Deep Neural Networks with Limited Memory. *ACM Trans. Math. Software* (2024).

[8] Olivier Beaumont, Lionel Eyraud-Dubois, and Alena Shilova. 2021. Efficient Combination of Rematerialization and Offloading for Training DNNs. In *Proceedings of the 34th Advances in Neural Information Processing Systems*.

[9] Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arXiv preprint arXiv:2004.05150* (2020).
[10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models Are Few-Shot Learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*.
[11] Benjamin Charlier, Jean Feydy, Joan Alexis Glaunes, François-David Collin, and Ghislain Durif. 2021. Kernel Operations on the GPU, with Autodiff, without Memory Overflows. *Journal of Machine Learning Research* (2021).
[12] Jianfei Chen, Lianmin Zheng, et al. 2021. ActNN: Reducing Training Memory Footprint via 2-Bit Activation Compressed Training. In *Proceedings of the 38th International Conference on Machine Learning*.
[13] Ping Chen, Zhuohong Deng, Ping Li, Shuibing He, Hongzi Zhu, Yi Zheng, Zhefeng Wang, Baoxing Huai, and Minyi Guo. 2025. Adacc: An Adaptive Framework Unifying Compression and Activation Recomputation for LLM Training. *arXiv preprint arXiv:2508.00806* (2025).
[14] Ping Chen, Shuibing He, Peiyi Hong, Xuechen Zhang, Siling Yang, Tong Wu, Gang Chen, and Xian-He Sun. 2026. NVREC: A High-Performance Recoverable Recommendation System for Non-Volatile Memory. *Transactions on Storage*.
[15] Ping Chen, Shuibing He, Xuechen Zhang, Shuaiben Chen, Peiyi Hong, Yanlong Yin, and Xian-He Sun. 2022. Accelerating Tensor Swapping in GPUs with Self-Tuning Compression. *IEEE Transactions on Parallel and Distributed Systems* (2022).
[16] Ping Chen, Shuibing He, Xuechen Zhang, Shuaiben Chen, Peiyi Hong, Yanlong Yin, Xian-He Sun, and Gang Chen. 2021. CSWAP: A Self-Tuning Compression Framework for Accelerating Tensor Swapping in GPUs. In *Proceedings of the IEEE International Conference on Cluster Computing*.
[17] Ping Chen, Wenjie Zhang, Shuibing He, Weijian Chen, Siling Yang, Kexin Huang, Yanlong Yin, Xuan Zhan, Yingjie Gu, Zhuwei Peng, et al. 2024. Optimizing Large Model Training through Overlapped Activation Recomputation. *arXiv preprint arXiv:2406.08756* (2024).
[18] Qiaoling Chen, Shenggui Li, Wei Gao, Peng Sun, Yonggang Wen, and Tianwei Zhang. 2025. SPPO: Efficient Long-Sequence LLM Training via Adaptive Sequence Pipeline Parallel Offloading. *arXiv preprint arXiv:2503.10377* (2025).
[19] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training Deep Nets with Sublinear Memory Cost. *arXiv preprint arXiv:1604.06174* (2016).
[20] Weiduo Chen, Xiaoshe Dong, Fan Zhang, Bowen Li, Yufei Wang, and Qiang Wang. 2025. ATP: Achieving Throughput Peak for DNN Training via Smart GPU Memory Management. *ACM Transactions on Architecture and Code Optimization* (2025).
[21] Xiaoming Chen, Danny Z Chen, and Xiaobo Sharon Hu. 2018. moDNN: Memory Optimal DNN Training on GPUs. In *Proceedings of the 2018 Design, Automation & Test in Europe Conference & Exhibition*.
[22] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. Prime: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. *ACM SIGARCH Computer Architecture News* (2016).
[23] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating Long Sequences with Sparse Transformers. *arXiv preprint arXiv:1904.10509* (2019).
[24] Brian Chmiel, Maxim Fishman, Ron Banner, and Daniel Soudry. 2025. FP4 All the Way: Fully Quantized Training of Large Language Models. In *Proceedings of the 39th Advances in Neural Information Processing Systems*.
[25] Minsik Cho and Daniel Brand. 2017. MEC: Memory-Efficient Convolution for Deep Neural Network. In *Proceedings of the 34th International Conference on Machine Learning*.
[26] CXL Consortium. 2020. *Compute Express Link Specification*. Technical Report. Compute Express Link Consortium.
[27] Pengcheng Dai, Jianlei Yang, Xucheng Ye, Xingzhou Cheng, Junyu Luo, Linghao Song, Yiran Chen, and Weisheng Zhao. 2020. SparseTrain: Exploiting Dataflow Sparsity for Efficient Convolutional Neural Networks Training. In *2020 57th ACM/IEEE Design Automation Conference*.
[28] Zheng Dang, Shuibing He, Peiyi Hong, Zhenxin Li, Xuechen Zhang, Xian-He Sun, and Gang Chen. 2022. NVAlloc: Rethinking heap metadata management in persistent memory allocators. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.
[29] Zheng Dang, Shuibing He, Xuechen Zhang, Peiyi Hong, Zhenxin Li, Xinyu Chen, Haozhe Song, Xian-He Sun, and Gang Chen. 2024. PMAlloc: A holistic approach to improving persistent memory allocation. *ACM Transactions on Computer Systems* (2024).
[30] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *Proceedings of the 12th International Conference on Learning Representations*.
[31] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Proceedings of the 35th Advances in Neural Information Processing Systems*.

[32] DeepSeek-AI. 2026. *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence.* Technical Report. DeepSeek-AI.
[33] Tim Dettmers, Mike Lewis, Sam Shleifer, and Luke Zettlemoyer. 2022. 8-Bit Optimizers via Block-Wise Quantization. In *Proceedings of the 10th International Conference on Learning Representations.*
[34] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers).*
[35] Yifan Ding, Nicholas Botzer, and Tim Weninger. 2021. HetSeq: Distributed GPU Training on Heterogeneous Infrastructure. In *Proceedings of the 35th AAAI Conference on Artificial Intelligence.*
[36] Henk Dreuning, Henri E Bal, and Rob V van Nieuwpoort. 2022. mCAP: Memory-Centric Partitioning for Large-Scale Pipeline-Parallel DNN Training. In *Proceedings of the 2022 European Conference on Parallel Processing.*
[37] Jiangfei Duan, Shuo Zhang, Zerui Wang, Lijuan Jiang, Wenwen Qu, Qinghao Hu, Guoteng Wang, Qizhen Weng, Hang Yan, Xingcheng Zhang, et al. 2026. Efficient Training of Large Language Models on Distributed Infrastructures: A Survey. *Vicinagearth* (2026).
[38] R. David Evans and Tor M. Aamodt. 2021. AC-GC: Lossy Activation Compression with Guaranteed Convergence. In *Proceedings of the 35th Advances in Neural Information Processing Systems.*
[39] Utku Evci, Trevor Gale, Jacob Menick, Pablo Samuel Castro, and Erich Elsen. 2020. Rigging the Lottery: Making All Tickets Winners. In *Proceedings of the 37th International Conference on Machine Learning.*
[40] Shiqing Fan, Yi Rong, Chen Meng, Zongyan Cao, Siyu Wang, Zhen Zheng, Chuan Wu, Guoping Long, Jun Yang, Lixue Xia, et al. 2021. DAPPLE: A Pipelined Data Parallel Approach for Training Large Models. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming.*
[41] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* (2022).
[42] Yanjie Gao, Yu Liu, Hongyu Zhang, Zhengxian Li, Yonghao Zhu, Haoxiang Lin, and Mao Yang. 2020. Estimating GPU Memory Consumption of Deep Learning Models. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering.*
[43] Anteneh Gebregiorgis, Hoang Anh Du Nguyen, Jintao Yu, Rajendra Bishnoi, Mottaqiallah Taouil, Francky Catthoor, and Said Hamdioui. 2022. A Survey on Memory-Centric Computer Architectures. *ACM Journal on Emerging Technologies in Computing Systems* (2022).
[44] Cong Guo, Chen Zhang, Jingwen Leng, Zihan Liu, Fan Yang, Yunxin Liu, Minyi Guo, and Yuhao Zhu. 2022. ANT: Exploiting Adaptive Numerical Data Type for Low-Bit Deep Neural Network Quantization. In *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture.*
[45] Cong Guo, Rui Zhang, Jiale Xu, Jingwen Leng, Zihan Liu, Ziyu Huang, Minyi Guo, Hao Wu, Shouren Zhao, Junping Zhao, et al. 2024. GMLake: Efficient and Transparent GPU Memory Defragmentation for Large-Scale DNN Training with Virtual Memory Stitching. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2.*
[46] Hongcan Guo, Haolang Lu, Guoshun Nan, Bolun Chu, Jialin Zhuang, Yuan Yang, Wenhao Che, Xinye Cao, Sicong Leng, Qimei Cui, and Xudong Jiang. 2026. Advancing Expert Specialization for Better MoE. In *Proceedings of the 39th Advances in Neural Information Processing Systems.*
[47] Yizhou Han, Chaohao Yang, Congliang Chen, Xingjian Wang, and Ruoyu Sun. 2025. Q-Adam-Mini: Memory-Efficient 8-Bit Quantized Optimizer for Large Language Model Training. In *Proceedings of the 3rd Workshop on Efficient Systems for Foundation Models at the 42nd International Conference on Machine Learning.*
[48] Cole Hawkins and Zheng Zhang. 2021. Bayesian Tensorized Neural Networks with Automatic Rank Selection. *Neurocomputing* (2021).
[49] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. 2022. FasterMoE: Modeling and Optimizing Training of Large-Scale Dynamic Pre-Trained Models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming.*
[50] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *Proceedings of the 29th IEEE Conference on Computer Vision and Pattern Recognition.*
[51] Shuibing He, Ping Chen, Shuaiben Chen, Zheng Li, Siling Yang, Weijian Chen, and Lidan Shou. 2022. Home: A Holistic GPU Memory Management Framework for Deep Learning. *IEEE Trans. Comput.* (2022).
[52] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* (1997).
[53] Abdelrahman Hosny, Marina Neseem, and Sherief Reda. 2021. Sparse Bitmap Compression for Memory-Efficient Training on the Edge. In *Proceedings of the Sixth ACM/IEEE Symposium on Edge Computing.*
[54] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *Proceedings of the 10th International Conference on Learning Representations.*

[55] Zhongzhe Hu, Junmin Xiao, Zheye Deng, Mingyi Li, Kewei Zhang, Xiaoyang Zhang, Ke Meng, Ninghui Sun, and Guangming Tan. 2022. MegTaiChi: Dynamic Tensor-Based Memory Management Optimization for DNN Training. In *Proceedings of the 36th ACM International Conference on Supercomputing*.

[56] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. 2017. Densely Connected Convolutional Networks. In *Proceedings of the 30th IEEE Conference on Computer Vision and Pattern Recognition*.

[57] Jun Huang, Zhen Zhang, Shuai Zheng, Feng Qin, and Yida Wang. 2024. DISTMM: Accelerating Distributed Multimodal Model Training. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation*.

[58] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism. In *Proceedings of the 33rd Conference on Neural Information Processing Systems*.

[59] Yuzhou Huang, Yapeng Jiang, Zicong Hong, Wuhui Chen, Bin Wang, Weixi Zhu, Yue Yu, and Zibin Zheng. 2025. Obscura: Concealing Recomputation Overhead in Training of Large Language Models with Bubble-Filling Pipeline Transformation. In *Proceedings of the 29th USENIX Annual Technical Conference*.

[60] Zixiao Huang, Junhao Hu, Hao Lin, Chunyang Zhu, Yueran Tang, Quanlu Zhang, Zhen Guo, Zhenhua Li, Shengen Yan, Zhenhua Zhu, Guohao Dai, and Yu Wang. 2026. STAlloc: Enhancing Memory Efficiency in Large-Scale Model Training with Spatio-Temporal Planning. In *Proceedings of the 21st European Conference on Computer Systems*.

[61] Akifumi Imanishi and Zijian Xu. 2024. A Heuristic for Periodic Memory Allocation with Little Fragmentation to Train Neural Networks. In *Proceedings of the 2024 ACM SIGPLAN International Symposium on Memory Management*.

[62] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Reza Yazdani Aminadabi, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2024. System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models. In *Proceedings of the 43rd ACM Symposium on Principles of Distributed Computing*.

[63] Animesh Jain, Amar Phanishayee, Jason Mars, Lingjia Tang, and Gennady Pekhimenko. 2018. Gist: Efficient Data Encoding for Deep Neural Network Training. In *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture*.

[64] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Joseph Gonzalez, Kurt Keutzer, and Ion Stoica. 2020. Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization. In *Proceedings of Machine Learning and Systems*.

[65] Yixin Ji, Yang Xiang, et al. 2025. Beware of Calibration Data for Pruning Large Language Models. In *Proceedings of the 13th International Conference on Learning Representations*.

[66] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825* (2023).

[67] Wenbin Jiang, Yang Ma, Bo Liu, Haikun Liu, Bing Bing Zhou, Jian Zhu, Song Wu, and Hai Jin. 2019. Layup: Layer-Adaptive and Multi-Type Intermediate-Oriented Memory Optimization for GPU-Based CNNs. *ACM Transactions on Architecture and Code Optimization* (2019).

[68] Hai Jin, Bo Liu, Wenbin Jiang, Yang Ma, Xuanhua Shi, Bingsheng He, and Shaofeng Zhao. 2018. Layer-Centric Memory Reuse and Data Migration for Extreme-Scale Deep Learning on Many-Core Architectures. *ACM Transactions on Architecture and Code Optimization* (2018).

[69] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. 2021. Highly Accurate Protein Structure Prediction with AlphaFold. *Nature* (2021).

[70] Dhiraj D. Kalamkar, Dheevatsa Mudigere, Naveen Mellempudi, Dipankar Das, Kunal Banerjee, Sasikanth Avancha, Dharma Teja Vooturi, Nataraj Jammalamadaka, Jianyu Huang, Hector Yuen, Jiyan Yang, Jongsoo Park, Alexander Heinecke, Evangelos Georganas, Sudarshan Srinivasan, Abhisek Kundu, Misha Smelyanskiy, Bharat Kaul, and Pradeep Dubey. 2019. A Study of BFLOAT16 for Deep Learning Training. *arXiv preprint arXiv:1905.12322* (2019).

[71] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361* (2020).

[72] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers Are RNNs: Fast Autoregressive Transformers with Linear Attention. In *Proceedings of the 37th International Conference on Machine Learning*.

[73] Taebum Kim, Hyoungjoo Kim, Gyeong-In Yu, and Byung-Gon Chun. 2023. BPipe: Memory-Balanced Pipeline Parallelism for Training Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning*.

[74] Diederik P Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *Proceedings of the 3rd International Conference on Learning Representations*.

[75] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. 2021. Dynamic Tensor Rematerialization. In *Proceedings of the 9th International Conference on Learning Representations*.

[76] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing Activation Recomputation in Large Transformer Models. *Proceedings of Machine Learning and Systems*.

[77] Ravi Kumar, Manish Purohit, Zoya Svitkina, Erik Vee, and Joshua Wang. 2019. Efficient Rematerialization for Deep Networks. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*.

[78] Mitsuru Kusumoto, Takuya Inoue, Gentaro Watanabe, Takuya Akiba, and Masanori Koyama. 2019. A Graph Theoretic Framework of Recomputation Algorithms for Memory-Efficient Backpropagation. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*.

[79] Ruihang Lai, Junru Shao, Siyuan Feng, Steven Lyubomirsky, Bohan Hou, Wuwei Lin, Zihao Ye, Hongyi Jin, Yuchen Jin, Jiawei Liu, et al. 2025. Relax: Composable Abstractions for End-to-End Dynamic Machine Learning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*.

[80] Zhiquan Lai, Shengwei Li, Xudong Tang, Keshi Ge, Weijie Liu, Yabo Duan, Linbo Qiao, and Dongsheng Li. 2023. Merak: An Efficient Distributed DNN Training Framework with Automated 3D Parallelism for Giant Foundation Models. *IEEE Transactions on Parallel and Distributed Systems* (2023).

[81] Mike Lasby, Anna Golubeva, et al. 2024. Dynamic Sparse Training with Structured Sparsity. In *Proceedings of the 12th International Conference on Learning Representations*.

[82] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2020. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. *arXiv preprint arXiv:2006.16668* (2020).

[83] Mike Lewis, Shruti Bhosale, Tim Dettmers, Naman Goyal, and Luke Zettlemoyer. 2021. Base Layers: Simplifying Training of Large, Sparse Models. In *Proceedings of the 38th International Conference on Machine Learning*.

[84] Dongsheng Li, Shengwei Li, Zhiquan Lai, Yongquan Fu, Xiangyu Ye, Lei Cai, and Linbo Qiao. 2023. A Memory-Efficient Hybrid Parallel Framework for Deep Neural Network Training. *IEEE Transactions on Parallel and Distributed Systems* (2023).

[85] Fanxin Li, Shixiong Zhao, Yuhao Qing, Jianyu Jiang, Xusheng Chen, and Heming Cui. 2025. PIPEMESH: Achieving Memory-Efficient Computation-Communication Overlap for Training Large Language Models. *IEEE Transactions on Parallel and Distributed Systems* (2025).

[86] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. 2023. Sequence Parallelism: Long Sequence Training from System Perspective. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.

[87] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. 2020. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. *Proceedings of the VLDB Endowment* (2020).

[88] Zhouyang Li, Yuliang Liu, Wei Zhang, Tailing Yuan, Bin Chen, and Chengru Song. 2025. SlimPipe: Memory-Thrifty and Efficient Pipeline Parallelism for Long-Context LLM Training. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*.

[89] Zhuohan Li, Siyuan Zhuang, Shiyuan Guo, Danyang Zhuo, Hao Zhang, Dawn Song, and Ion Stoica. 2021. TeraPipe: Token-Level Pipeline Parallelism for Training Large-Scale Language Models. In *Proceedings of the 38th International Conference on Machine Learning*.

[90] Xinyu Lian, Masahiro Tanaka, Olatunji Ruwase, and Minjia Zhang. 2026. SuperOffload: Unleashing the Power of Large-Scale LLM Training on Superchips. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.

[91] Peng Liang, Yu Tang, Xiaoda Zhang, Youhui Bai, Teng Su, Zhiquan Lai, Linbo Qiao, and Dongsheng Li. 2023. A survey on auto-parallelism of large-scale deep learning training. *IEEE Transactions on Parallel and Distributed Systems* (2023).

[92] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv preprint arXiv:2405.04434* (2024).

[93] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024).

[94] Guodong Liu, Youshan Miao, Zhiqi Lin, Xiaoxiang Shi, Saeed Maleki, Fan Yang, Yungang Bao, and Sa Wang. 2024. Aceso: Efficient Parallel DNN Training through Iterative Bottleneck Alleviation. In *Proceedings of the 19th European Conference on Computer Systems*.

[95] Hao Liu and Pieter Abbeel. 2023. Blockwise Parallel Transformers for Large Context Models. In *Proceedings of the 36th Advances in Neural Information Processing Systems*.
[96] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. In *Proceedings of the 36th Advances in Neural Information Processing Systems*.
[97] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2024. RingAttention with Blockwise Transformers for Near-Infinite Context. In *Proceedings of the 12th International Conference on Learning Representations*.
[98] Weijian Liu, Mingzhen Li, Guangming Tan, and Weile Jia. 2025. Mario: Near Zero-Cost Activation Checkpointing in Pipeline Parallelism. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*.
[99] Xiaoxuan Liu, Lianmin Zheng, Dequan Wang, Yukuo Cen, Weize Chen, Xu Han, Jianfei Chen, Zhiyuan Liu, Jie Tang, Joey Gonzalez, Michael Mahoney, and Alvin Cheung. 2022. GACT: Activation Compressed Training for Generic Network Architectures. In *Proceedings of the 39th International Conference on Machine Learning*.
[100] Amir M. Mansourian, Rozhan Ahmadi, Masoud Ghafouri, Amir Mohammad Babaei, Elaheh Badali Golezani, Zeynab Yasamani Ghamchi, Vida Ramezanian, Alireza Taherian, Kimia Dinashi, Amirali Miri, and Shohreh Kasaei. 2025. A Comprehensive Survey on Knowledge Distillation. *Transactions on Machine Learning Research* (2025).
[101] Avinash Maurya, Robert Underwood, M Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. 2024. Datastates-LLM: Lazy Asynchronous Checkpointing for Large Language Models. In *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing*.
[102] Avinash Kumar Maurya, M Mustafa Rafique, Franck Cappello, and Bogdan Nicolae. 2025. MLP-Offload: Multi-Level, Multi-Path Offloading for LLM Pre-Training to Break the GPU Memory Wall. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*.
[103] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. 2022. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism. *Proceedings of the VLDB Endowment* (2022).
[104] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. In *Proceedings of the 6th International Conference on Learning Representations*.
[105] Zhangqiang Ming, Rui Wang, Yuchong Hu, Yuanhao Shu, Wenxiang Zhou, Xinjue Zheng, and Dan Feng. 2026. SSFusion: Tensor fusion with selective sparsification for efficient distributed DNN training. In *Proceedings of the 2026 IEEE 42nd International Conference on Data Engineering*.
[106] Hesham Mostafa and Xin Wang. 2019. Parameter Efficient Training of Deep Convolutional Neural Networks by Dynamic Sparse Reparameterization. In *Proceedings of the 36th International Conference on Machine Learning*.
[107] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*.
[108] Deepak Narayanan, Amar Phanishayee, Kaiyu Shi, Xie Chen, and Matei Zaharia. 2021. Memory-Efficient Pipeline-Parallel DNN Training. In *Proceedings of the 38th International Conference on Machine Learning*.
[109] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. 2020. Heterogeneity-Aware Cluster Scheduling Policies for Deep Learning Workloads. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation*.
[110] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*.
[111] NVIDIA. 2024. *DGX GB200 SuperPOD Reference Architecture: Scalable Infrastructure.* Technical Report. NVIDIA.
[112] NVIDIA. 2024. *GPUDirect Storage Design Guide.* Technical Report. NVIDIA.
[113] NVIDIA Corporation. 2017. *NVIDIA Tesla V100 GPU Architecture: The World's Most Advanced Data Center GPU.* Technical Report. NVIDIA Corporation.
[114] NVIDIA Corporation. 2022. *NVIDIA Grace Hopper Superchip Architecture.* Technical Report. NVIDIA.
[115] Yunjie Pan, Yongyi Yang, Hanmei Yang, and Scott Mahlke. 2026. SNIP: An Adaptive Mixed Precision Framework for Subbyte Large Language Model Training. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.
[116] Jay H Park, Gyeongchan Yun, M Yi Chang, Nguyen T Nguyen, Seungmin Lee, Jaesik Choi, Sam H Noh, and Young-ri Choi. 2020. HetPipe: Enabling Large DNN Training on (Whimpy) Heterogeneous GPU Clusters through Integration of Pipelined Model Parallelism and Data Parallelism. In *Proceedings of the 2020 USENIX Annual Technical Conference*.
[117] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning

Library. In *Proceedings of the 32nd Advances in Neural Information Processing Systems*.

[118] Houwen Peng, Kan Wu, Yixuan Wei, Guoshuai Zhao, Yuxiang Yang, Ze Liu, Yifan Xiong, Ziyue Yang, Bolin Ni, Jingcheng Hu, Ruihang Li, Miaosen Zhang, Chen Li, Jia Ning, Ruizhe Wang, Zheng Zhang, Shuguang Liu, Joe Chau, Han Hu, and Peng Cheng. 2023. FP8-LM: Training FP8 Large Language Models. *arXiv preprint arXiv:2310.18313* (2023).

[119] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. 2020. Capuchin: Tensor-Based GPU Memory Management for Deep Learning. In *Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems*.

[120] Xuan Peng, Xuanhua Shi, Haolin Zhang, Yunfei Zhao, and Xuehai Qian. 2025. DawnPiper: A Memory-Scalable Pipeline Parallel Training Framework. *arXiv preprint arXiv:2505.05856* (2025).

[121] Cory Perry and Nikolay Sakharnykh. 2020. *Introducing Low-Level GPU Virtual Memory Management*. Technical Report.

[122] Matthew E Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. Deep Contextualized Word Representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*.

[123] Igor Polyakov, Alexey Dukhanov, and Egor Spirin. 2025. TAGC: Optimizing Gradient Communication in Distributed Transformer Training. In *Proceedings of the 5th Workshop on Machine Learning and Systems*.

[124] Zihan Qiu, Zeyu Huang, Bo Zheng, Kaiyue Wen, Zekun Wang, Rui Men, Ivan Titov, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. Demons in the Detail: On Implementing Load Balancing Loss for Training Specialized Mixture-of-Expert Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*.

[125] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning Transferable Visual Models from Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning*.

[126] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. *Language Models Are Unsupervised Multitask Learners*. Technical Report. OpenAI.

[127] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* (2020).

[128] Md Aamir Raihan and Tor M. Aamodt. 2020. Sparse Weight Activation Training. In *Advances in Neural Information Processing Systems*.

[129] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory Optimizations toward Training Trillion Parameter Models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*.

[130] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. Zero-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis*.

[131] Jie Ren, Jiaolin Luo, Kai Wu, Minjia Zhang, Hyeran Jeon, and Dong Li. 2021. Sentinel: Efficient Tensor Migration and Allocation on Heterogeneous Memory Systems for Deep Learning. In *Proceedings of the 27th IEEE International Symposium on High-Performance Computer Architecture*.

[132] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. Zero-Offload: Democratizing Billion-Scale Model Training. In *Proceedings of the 2021 USENIX Annual Technical Conference*.

[133] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W Keckler. 2016. vDNN: Virtualized Deep Neural Networks for Scalable, Memory-Efficient Neural Network Design. In *Proceedings of the 2016 49th Annual IEEE/ACM International Symposium on Microarchitecture*.

[134] Minsoo Rhu, Mike O'Connor, Niladrish Chatterjee, Jeff Pool, Youngeun Kwon, and Stephen W Keckler. 2018. Compressing DMA Engine: Leveraging Activation Sparsity for Training Deep Neural Networks. In *Proceedings of the 2018 IEEE International Symposium on High Performance Computer Architecture*.

[135] Herbert Robbins and Sutton Monro. 1951. A Stochastic Approximation Method. *The Annals of Mathematical Statistics* (1951).

[136] Dayana Savostianova, Emanuele Zangrando, Gianluca Ceruti, and Francesco Tudisco. 2023. Robust Low-Rank Training via Approximate Orthonormal Constraints. In *Proceedings of the 37th Advances in Neural Information Processing Systems*.

[137] Steffen Schotthöfer, H. Lexie Yang, and Stefan Schnake. 2025. Dynamical Low-Rank Compression of Neural Networks with Robustness under Adversarial Attacks. In *Proceedings of the 39th Advances in Neural Information Processing Systems*.

[138] Steffen Schotthöfer, Emanuele Zangrando, et al. 2022. Low-Rank Lottery Tickets: Finding Efficient Low-Rank Neural Networks via Matrix Differential Equations. In *Proceedings of the 36th Advances in Neural Information Processing Systems*.

[139] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision. In *Proceedings of the 37th Advances in Neural Information Processing Systems*.

[140] Yara Shamshoum, Nitzan Hodos, Yuval Sieradzki, and Assaf Schuster. 2025. COmpACt: Compressed Activations for Memory-Efficient LLM Training. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*.

[141] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *Proceedings of the 5th International Conference on Learning Representations*.

[142] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[143] SB Shriram, Anshuj Garg, and Purushottam Kulkarni. 2019. Dynamic Memory Management for GPU-Based Training of Deep Neural Networks. In *Proceedings of the 2019 IEEE International Parallel and Distributed Processing Symposium*.

[144] Huiyao Shu, Ang Wang, Ziji Shi, Hanyu Zhao, Yong Li, and Lu Lu. 2023. Roam: Memory-Efficient Large DNN Training via Optimized Operator Ordering and Memory Layout. *arXiv preprint arXiv:2310.19295* (2023).

[145] Patrick Siegl, Rainer Buchty, and Mladen Berekovic. 2016. Data-Centric Computing Frontiers: A Survey on Processing-in-Memory. In *Proceedings of the Second International Symposium on Memory Systems*.

[146] Benoit Steiner, Mostafa Elhoushi, Jacob Kahn, and James Hegarty. 2023. MODeL: Memory Optimizations for Deep Learning. In *Proceedings of the 40th International Conference on Machine Learning*.

[147] Yang Sui, Miao Yin, et al. 2024. ELRT: Efficient Low-Rank Training for Compact Convolutional Neural Networks. *arXiv preprint arXiv:2401.10341* (2024).

[148] Zhenbo Sun, Huanqi Cao, Yuanwei Wang, Guanyu Feng, Shengqi Chen, Haojie Wang, and Wenguang Chen. 2024. AdaPipe: Optimizing Pipeline Parallelism with Adaptive Recomputation and Partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*.

[149] Zhenbo Sun, Shengqi Chen, Yuanwei Wang, Jian Sha, Guanyu Feng, and Wenguang Chen. 2025. MePipe: Democratizing LLM Training with Memory-Efficient Slice-Level Pipeline Scheduling on Cost-Effective Accelerators. In *Proceedings of the 20th European Conference on Computer Systems*.

[150] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alexander Alemi. 2017. Inception-V4, Inception-ResNet and the Impact of Residual Connections on Learning. In *Proceedings of the 31st AAAI Conference on Artificial Intelligence*.

[151] Jakub M Tarnawski, Deepak Narayanan, and Amar Phanishayee. 2021. Piper: Multidimensional Planner for DNN Parallelization. In *Proceedings of the 34th Advances in Neural Information Processing Systems*.

[152] Kimi Team, Yifan Bai, Yiping Bao, Y Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, et al. 2025. Kimi K2: Open Agentic Intelligence. *arXiv preprint arXiv:2507.20534* (2025).

[153] Meituan LongCat Team, Bairui Wang, Bin Xiao, Bo Zhang, Bolin Rong, Borun Chen, Chang Wan, Chao Zhang, Chen Huang, Chen Chen, et al. 2025. LongCat-Flash-Omni Technical Report. *arXiv preprint arXiv:2511.00279* (2025).

[154] Kaiyuan Tian, Linbo Qiao, Baihui Liu, Gongqingjian Jiang, Shanshan Li, and Dongsheng Li. 2026. A Survey on Memory-Efficient Transformer-Based Model Training in AI for Science. *Frontiers of Computer Science* (2026).

[155] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint arXiv:2302.13971* (2023).

[156] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. LLaMA 2: Open Foundation and Fine-Tuned Chat Models. *arXiv preprint arXiv:2307.09288* (2023).

[157] Taegeon Um, Byungsoo Oh, Minyoung Kang, Woo-Yeon Lee, Goeun Kim, Dongseob Kim, Youngtaek Kim, Mohd Muzzammil, and Myeongjae Jeon. 2024. Metis: Fast Automatic Distributed Training on Heterogeneous GPUs. In *Proceedings of the 2024 USENIX Annual Technical Conference*.

[158] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*.

[159] Mohamed Wahib, Haoyu Zhang, Truong Thao Nguyen, Aleksandr Drozd, Jens Domke, Lingqi Zhang, Ryousei Takano, and Satoshi Matsuoka. 2020. Scaling Distributed Deep Learning Workloads beyond the Memory Capacity with Karma. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage, and Analysis*.

[160] Xinyi Wan, Penghui Qi, Guangxing Huang, Min Lin, and Jialin Li. 2025. PipeOffload: Improving Scalability of Pipeline Parallelism with Memory Optimization. In *Proceedings of the 42nd International Conference on Machine Learning*.

[161] Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, Mosharaf Chowdhury, and Mi Zhang. 2024. Efficient Large Language Models: A Survey. *Transactions on Machine Learning Research* (2024).

[162] Hongyu Wang, Shuming Ma, et al. 2023. BitNet: Scaling 1-Bit Transformers for Large Language Models. *arXiv preprint arXiv:2310.11453* (2023).

[163] Lean Wang, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. 2024. Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts. *arXiv preprint arXiv:2408.15664* (2024).

[164] Linnan Wang, Jinmian Ye, Yiyang Zhao, Wei Wu, Ang Li, Shuaiwen Leon Song, Zenglin Xu, and Tim Kraska. 2018. Superneurons: Dynamic GPU Memory Management for Training Deep Neural Networks. In *Proceedings of the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*.

[165] Naigang Wang, Jungwook Choi, et al. 2018. Training Deep Neural Networks with 8-Bit Floating Point Numbers. In *Proceedings of the 32nd Advances in Neural Information Processing Systems*.

[166] Shiju Wang, Yujie Wang, Ao Sun, Fangcheng Fu, Zijian Zhu, Bin Cui, Xu Han, and Kaisheng Ma. 2025. Data-Centric Elastic Pipeline Parallelism for Efficient Long-Context LLM Training. *arXiv preprint arXiv:2509.21275* (2025).

[167] Yujie Wang, Youhe Jiang, Xupeng Miao, Fangcheng Fu, Shenhan Zhu, Xiaonan Nie, Yaofeng Tu, and Bin Cui. 2024. Improving Automatic Parallel Training via Balanced Memory Workload Optimization. *IEEE Transactions on Knowledge and Data Engineering* (2024).

[168] Zheng Wang, Anna Cai, Xinfeng Xie, Zaifeng Pan, Yue Guan, Weiwei Chu, Jie Wang, Li Shikai, Jianyu Huang, Chris Cai, et al. 2025. WLB-LLM: Workload-Balanced 4D Parallelism for Large Language Model Training. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation*.

[169] Zehua Wang, Junmin Xiao, Xiaochuan Deng, Huibing Wang, Hui Ma, Mingyi Li, Yunfei Pang, and Guangming Tan. 2026. T-Control: An Efficient Dynamic Tensor Rematerialization System for DNN Training. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems*.

[170] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*.

[171] Kun Wu, Jeongmin Brian Park, Xiaofan Zhang, Mert Hidayetoğlu, Vikram Sharma Mailthody, Sitao Huang, Steve Lumetta, and Wen-mei Hwu. 2025. SSDTrain: An Activation Offloading Framework to SSDs for Faster Large Language Model Training. In *Proceedings of the 2025 62nd ACM/IEEE Design Automation Conference*.

[172] Tong Wu, Shuibing He, Jianxin Zhu, Weijian Chen, Siling Yang, Ping Chen, Yanlong Yin, Xuechen Zhang, Xian-He Sun, and Gang Chen. 2024. AUTOHET: An Automated Heterogeneous ReRAM-Based Accelerator for DNN Inference. In *Proceedings of the 53rd International Conference on Parallel Processing*.

[173] Haocheng Xi, Han Cai, Ligeng Zhu, Yao Lu, Kurt Keutzer, Jianfei Chen, and Song Han. 2025. COAT: Compressing Optimizer States and Activations for Memory-Efficient FP8 Training. In *Proceedings of the 13th International Conference on Learning Representations*.

[174] Haocheng Xi, ChangHao Li, Jianfei Chen, and Jun Zhu. 2023. Training Transformers with 4-Bit Integers. In *Proceedings of the 37th Advances in Neural Information Processing Systems*.

[175] Dong Xu, Yuan Feng, Kwangsik Shin, Daewoo Kim, Hyeran Jeon, and Dong Li. 2024. Efficient Tensor Offloading for Large Deep-Learning Model Training Based on Compute Express Link. In *Proceedings of the SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*.

[176] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388* (2025).

[177] Bowen Yang, Jian Zhang, Jonathan Li, Christopher Ré, Christopher Aberger, and Christopher De Sa. 2021. PipeMare: Asynchronous Pipeline Parallel DNN Training. *Proceedings of Machine Learning and Systems* (2021).

[178] Donglin Yang and Dazhao Cheng. 2020. Efficient GPU Memory Management for Nonlinear DNNs. In *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing*.

[179] Siling Yang, Weijian Chen, Xuechen Zhang, Shuibing He, Yanlong Yin, and Xian-He Sun. 2021. AUTO-PRUNE: Automated DNN pruning and mapping for ReRAM-based accelerator. In *Proceedings of the 35th ACM International Conference on Supercomputing*.

[180] Siling Yang, Shuibing He, Hexiao Duan, Weijian Chen, Xuechen Zhang, Tong Wu, and Yanlong Yin. 2023. APQ: Automated DNN pruning and quantization for ReRAM-based accelerators. *IEEE Transactions on Parallel and Distributed Systems* (2023).

[181] Siling Yang, Shuibing He, Wenjiong Wang, Yanlong Yin, Tong Wu, Weijian Chen, Xuechen Zhang, Xian-He Sun, and Dan Feng. 2025. GOPIM: GCN-Oriented Pipeline Optimization for PIM Accelerators. In *Proceedings of the 2025 IEEE International Symposium on High Performance Computer Architecture*.

[182] Zhiyi Yao, Zuning Liang, Yuedong Xu, Jin Zhao, Jessie Hui Wang, and Tong Li. 2025. MemFerry: A Fast and Memory Efficient Offload Training Framework with Hybrid GPU Computation. In *Proceedings of the 2025 IEEE International Conference on Computer Communications*.

[183] Xiangyu Ye, Zhiquan Lai, Shengwei Li, Lei Cai, Ding Sun, Linbo Qiao, and Dongsheng Li. 2021. Hippie: A Data-Paralleled Pipeline Approach to Improve Memory-Efficiency and Scalability for Large DNN Training. In *Proceedings of the 50th International Conference on Parallel Processing*.

[184] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, et al. 2020. Big Bird: Transformers for Longer Sequences. In *Proceedings of the 33rd Advances in Neural Information Processing Systems*.

[185] Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, et al. 2025. GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models. *arXiv preprint arXiv:2508.06471* (2025).

[186] Mingshu Zhai, Jiaao He, Zixuan Ma, Zan Zong, Runqing Zhang, and Jidong Zhai. 2023. SmartMoE: Efficiently Training Sparsely-Activated Models through Combining Offline and Online Parallelization. In *Proceedings of the 2023 USENIX Annual Technical Conference*.

[187] Jianhao Zhang, Shihan Ma, Peihong Liu, and Jinhui Yuan. 2023. COOP: Memory Is Not a Commodity. In *Proceedings of the 36th Advances in Neural Information Processing Systems*.

[188] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. OPT: Open Pre-Trained Transformer Language Models. *arXiv preprint arXiv:2205.01068* (2022).

[189] Tianyi Zhang and Anshumali Shrivastava. 2025. LeanQuant: Accurate and Scalable Large Language Model Quantization with Loss-Error-Aware Grid. In *Proceedings of the 13th International Conference on Learning Representations*.

[190] Zheng Zhang, Yaqi Xia, Hulin Wang, Donglin Yang, Chuang Hu, Xiaobo Zhou, and Dazhao Cheng. 2024. MPMoE: Memory Efficient MoE for Pre-Trained Models with Adaptive Pipeline Parallelism. *IEEE Transactions on Parallel and Distributed Systems* (2024).

[191] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. 2024. GALOre: Memory-Efficient LLM Training by Gradient Low-Rank Projection. In *Proceedings of the 41st International Conference on Machine Learning*.

[192] Pinxue Zhao, Hailin Zhang, Fangcheng Fu, Xiaonan Nie, Qibin Liu, Fang Yang, Yuanbo Peng, Dian Jiao, Shuaipeng Li, Jinbao Xue, et al. 2025. MEMO: Fine-Grained Tensor Management for Ultra-Long Context LLM Training. In *Proceedings of the ACM on Management of Data*.

[193] Shixiong Zhao, Fanxin Li, Xusheng Chen, Xiuxian Guan, Jianyu Jiang, Dong Huang, Yuhao Qing, Sen Wang, Peng Wang, Gong Zhang, et al. 2021. VPipe: A Virtualized Acceleration System for Achieving Efficient and Scalable Pipeline Parallel DNN Training. *IEEE Transactions on Parallel and Distributed Systems* (2021).

[194] Xunyi Zhao, Théotime Le Hellard, Lionel Eyraud-Dubois, Julia Gusak, and Olivier Beaumont. 2023. Rockmate: An Efficient, Fast, Automatic and Generic Tool for Re-Materialization in PyTorch. In *Proceedings of the 40th International Conference on Machine Learning*.

[195] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, et al. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proceedings of the VLDB Endowment* (2023).

[196] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P Xing, et al. 2022. Alpa: Automating Inter-and Intra-Operator Parallelism for Distributed Deep Learning. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation*.

[197] Quan Zhou, Haiquan Wang, Xiaoyan Yu, Cheng Li, Youhui Bai, Feng Yan, and Yinlong Xu. 2023. MPress: Democratizing Billion-Scale Model Training on Multi-GPU Servers via Memory-Saving Inter-Operator Parallelism. In *Proceedings of the 2023 IEEE International Symposium on High-Performance Computer Architecture*.

[198] Bohan Zhuang, Jing Liu, Zizheng Pan, Haoyu He, Yuetian Weng, and Chunhua Shen. 2023. A Survey on Efficient Training of Transformers. *arXiv preprint arXiv:2302.01107* (2023).

[199] Zan Zong, Li Lin, Leilei Lin, Lijie Wen, and Yu Sun. 2023. STR: Hybrid Tensor Re-Generation to Break Memory Wall for DNN Training. *IEEE Transactions on Parallel and Distributed Systems* (2023).

[200] Pengfei Zuo, Huimin Lin, Junbo Deng, Nan Zou, Xingkun Yang, Yingyu Diao, Weifeng Gao, Ke Xu, Zhangyu Chen, Shirui Lu, et al. 2025. Serving Large Language Models on Huawei CloudMatrix384. *arXiv preprint arXiv:2506.12708* (2025).